AccessMatrix Common Deployment Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - AccessMatrix Common Deployment Guide

This guide contains instructions on the deployment and maintenance of the AccessMatrix system.

Intended Reader

It is intended for system administrators to set up, operate and maintain the AccessMatrix system.

Document Scope

This document provides instructions on the various installation and deployment methods for the AccessMatrix system. For in-depth details and instructions on the cloud deployment methods for the AccessMatrix system, refer to the respective section of the *AccessMatrix Cloud Deployment Guide*.

For feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- *AccessMatrix Release Notes*
- *AccessMatrix Common Administration Guide*
- *AccessMatrix Cloud Deployment Guide*

2. Introduction to AccessMatrix Deployment

This page provides an overview of the AccessMatrix deployment process.

AccessMatrix Common Deployment

Upon successful setup of a new AccessMatrix security registry using the default database setup/initialize script, three initial administrators will be created in the AccessMatrix system:

- "amadmin"
- "amadmin1"
- "backupadmin"

Refer to the *AccessMatrix Common Administration Guide* for a detailed explanation on these administrators in terms of admin roles as well as admin rights assigned.

Each individual AccessMatrix product such as USO has its own specific components or modules. As a result, there are certain dependencies between the various components of AccessMatrix system. These dependencies are shown in the following table:

Component	Internal Dependency	External Dependency
Security Registry	- None -	JDBC-Compliant Database
AccessMatrix (AM) Server	Security Registry	Web App Server
Web Applications (USO SSF, USO SSO etc.)	AccessMatrix (AM) Server	Web App Server
USO Client	USO Agent	- None -

Note: The term *<ACMATRIX_ROOT>* will be used to refer to the root or installation directory of AccessMatrix on both Windows and Unix platforms.

3. Installation and Deployment

The following pages provide detailed information on the installation and deployment of the AccessMatrix system:

- System Requirements
- AccessMatrix Installation
- AccessMatrix Server as Docker Container
- AccessMatrix Server Connects to Redis Server
- AccessMatrix Advanced Installations
- Special Considerations
- Advanced Configurations

3.1. System Requirements

There are certain system requirements in order to be able to utilize the AccessMatrix system. The following section describes those.

Pre-installation Preparation and Prerequisites

- **AccessMatrix Service License Key**
    - By default, AccessMatrix Server comes with a 30-day evaluation license. After this period, a valid license key will be required to ensure proper server operation.
    - For production deployment, a license key needs to be deployed for each host on which the AccessMatrix Server is to be installed.
- **Server Security Settings**

  It is vital to ensure that the server components are installed on servers that are "hardened" and connected to the network segment protected by the firewall.
- **Firewall Port Opening**

  Various components of AccessMatrix communicate among themselves through TCP ports. Thus, ensure that the following firewall ports are opened prior to the installation:

Source Component	Destination Component	Default Destination Port (TCP)
Client Web Browser	AM Server (Admin Console)	8080 (Non-SSL - Tomcat) / 8443 (SSL - Tomcat)
Applications calling AM Server API	AM Server (API)	8080 (Non-SSL - Tomcat) / 8443 (SSL with mTLS - Tomcat)
AccessMatrix	Security Registry	3306 (MySQL) / 1433 (MS SQL) / 1521 (Oracle)

- **Server Time Synchronization**

  AccessMatrix servers require time synchronization across servers, as some of the modules are time-sensitive.

Note: Using Network Time Protocol (NTP) servers is highly recommended.

3.2. AccessMatrix Installation

This section contains the steps to install AccessMatrix Dispersed system using a dispersed database server. Under a typical installation of AccessMatrix Dispersed system, the administrator is not required to make any changes to the system except to set up the AccessMatrix system to integrate with the dispersed database server that hosts the security registry.

For performance and security reasons, it is recommended to install the AccessMatrix system on a dedicated machine. Here are the general steps to set up AccessMatrix Dispersed System:

1. Run the AccessMatrix Installer to install AccessMatrix.
2. Run the database script to create/update the AccessMatrix security registry.
3. Modify the relevant AccessMatrix system properties file to integrate with the desired database server that hosts the security registry.
4. Start the AccessMatrix system.
5. Log in to AccessMatrix Admin Console to set up the security policies, applications, users, and other configurations.

Download

Download the AccessMatrix installation package or acquire CD-Image from the system service provider.

Installation

This section contains the steps of installing the AccessMatrix system using a database server.

Precaution - Steps Required Prior to Installation

- **New Installation of AccessMatrix System**
  For a new installation of AccessMatrix system, i.e. there is not an existing version of AccessMatrix installed on the server, administrator may proceed to the Install on Windows Platform section for installation on Windows or Install on Unix Platform for installation on Unix.
- **Upgrade Installation of AccessMatrix System**
  For an upgrade installation of AccessMatrix system, i.e. there is an existing version of AccessMatrix installed on the server, the administrator is advised to uninstall the existing version of AccessMatrix prior to installing the new version.

Install on Windows Platform

Installation on Windows is done by running the AccessMatrix Installer.

To install, perform the following steps:

1. Double-click on the .exe application file to run the AccessMatrix installer, e.g. **Setup.exe**.
2. Click **Next** to proceed with the installation and the **License Agreement** screen will be displayed.
3. Accept the terms of the AccessMatrix license agreement and click **Next** to continue.
4. Select the directory to which the AccessMatrix will be installed. This will be the installation root or base directory, i.e. <*ACMATRIX_ROOT*>.
   All the necessary files will be installed in this root directory. The default root directory is *C:\Program Files(x86)\AccessMatrix*.
   Click **Browse** to change the default root directory, or **Next** to accept the default root directory and continue.
5. Select the components to be installed and deselect the components not to be installed as described below:
   - **AccessMatrix Server** (MUST SELECT) inclusive of the core server, i.e. AccessMatrix Server and Apache Tomcat Web application server.
   - **USO Trainer** (OPTIONAL) is an independent component that is used to train the application's **Login** and **Change Password** screens. It can be installed separately by itself on another computer.
   - **AccessMatrix Web Archive** (OPTIONAL) will generate the AccessMatrix Web Archive that encompasses AccessMatrix Server and AccessMatrix Web Applications such as USO Agent, Admin Console, etc.
     Click **Next**.
6. Specify the datasource name or accept the default value suggested and click **Next** to continue.
7. Specify a unique AM Server ID or accept the default value suggested and click **Next** to continue.

> **Note:** As mentioned in the comment, this unique AccessMatrix Server ID is used to identify the instance of the AccessMatrix Server for licensing and auditing purposes during AccessMatrix Server starts up. If a matching instance is not found, the AccessMatrix Server will fail to start. Once installed, the AccessMatrix Server ID should not be changed unnecessarily.

8. Review the installation settings and then click **Next** to start copying files or **Back** to change any settings. Clicking **Next** will allow the installer to start copying files to the designated root directory.
9. Having copied the installation files, if the USO Trainer component was selected during the installation, the installer will prompt the user to install USO Trainer 5.0.
    a. Click **Next** on the **USO Trainer setup** dialog box.
    b. Select the directory that USO Trainer 5.0 will be installed in. Click **Browse** to change the default directory, or **Next** to accept the default directory and continue.
    c. Click **Install** to start the installation, or **Back** to change any settings. Clicking **Install** will allow the installer to start copying files to the designated directory.
    d. Upon successful installation of USO Trainer, the **Complete Setup** screen will be displayed. Click **Finish** to continue and the installer will continue to copy the rest of the installation files of AccessMatrix system.

    > **Note:** If the USO Trainer component is not selected during installation, the above steps will be skipped.

10. Upon successful installation, the AccessMatrix complete setup screen is displayed. Click **Finish** and an information dialog box is displayed containing the AccessMatrix service information.
11. Click **OK** to exit the installation wizard. If applicable, close the **Readme** window to exit completely.
12. Start AccessMatrix server. The administrator can now proceed to operate in the AccessMatrix system.

> **Note:**
>
> - By default, AccessMatrix service is configured to run using Local Service user. AccessMatrix installer has automatically given permission to Local Service user to access the installation directory. However, if there is a need to access additional directory or file outside the AccessMatrix installation directory, you need to grant permission to Local Service user to access the directory or file. You may also create a specific service user solely for running AccessMatrix and grant necessary permissions to that user.
> - If you need to modify the user used to run the AccessMatrix service, you may do so by running *<ACMATRIX_ROOT>\tomcat\bin\am5w.exe* . Alternatively, you may run **Monitor AccessMatrix Server**, right-click on the **AccessMatrix** monitoring icon in the taskbar, and click on **Configure**.

	○ Running using Local System will also work, but Local Service is recommended for better security as it has lesser permission.

Install on Unix Platform

Installation on Unix is done by extracting the AccessMatrix compressed archive file.

Pre-Installation Tasks for Unix

Log in as the "root" user on the AccessMatrix server machine, and perform the following tasks to set up the AccessMatrix environment:

- **Create Unix Group for AccessMatrix Software**

  Use the admin tool or groupadd utility to create a group named "acmatrix".

- **Create Unix Account to own AccessMatrix Software**

  Use the admin tool or useradd utility to create a user account named "acmatrix" with the following properties:

Field Name	Property
Login Name	acmatrix
Primary Group	acmatrix
Home Directory	/opt/acmatrix
Login Shell	bash

  The "acmatrix" account is the user account that owns the AccessMatrix software after installation. The */opt/acmatrix/* directory is referred as the <*ACMATRIX_ROOT*> directory.

- **Create AccessMatrix Installation Directory on Unix**

  Log in as "root" to the server and create the directory with the appropriate ownership shown in the table below:

Installation Directory	Ownership
/opt/acmatrix	acmatrix:acmatrix

  The following are example commands for the above:

```
# mkdir –p /opt/acmatrix
# chmod 755 /opt/acmatrix
# chown –R acmatrix:acmatrix /opt/acmatrix
```

Pre-Installation Tasks to Perform as the "acmatrix" User

Log in as the "acmatrix" account on the AccessMatrix Server, and perform the following tasks where appropriate:

- **Set Permissions for File Creation**
    1. Set umask to "022" for the "acmatrix" account to ensure that group and others have read and execute permissions but not write permission on the installed files.
    2. Enter the `umask` command to check the current setting.
    3. If the `umask` command does not return 022, set it in the "acmatrix" login profile (e.g. .profile, .login, or .bash_profile), then execute the command `$ umask 022` .

> **Note:**
>
> - User can check the login profile used by checking the user's login shell.
> - To check the login shell, perform the command `$ echo $SHELL`.
> - For Bash shell, the login profile is stored at .bash_profile.

- **Set Environment Variable**

1. Before starting the Installer, set the following environment variable by editing the login profile of the "acmatrix" account accordingly.
2. Set JAVA_HOME (the home directory containing the JDK/JRE used to run AccessMatrix) to:
   ```
   JAVA_HOME=<JAVA_Installation_Directory>
   export JAVA_HOME
   ```
3. Verify the environment variable setting by logging out and logging in again as "acmatrix" account and then run the command `$ which java` .
   If the command can be executed successfully, the system should return the version information of the JDK/JRE.

Installation Steps

For installation of AccessMatrix system on Unix, the AccessMatrix Installer is compiled as a compressed archive file in the form of a ZIP archive (.zip). To install, perform the following steps:

1. Log in as the "acmatrix" account.
2. Browse the AccessMatrix CD to look for the AccessMatrix Installer ZIP archive.
3. Extract the AccessMatrix Installer ZIP archive to the *<ACMATRIX_ROOT>* directory, by entering the command

`$ unzip <AccessMatrix-ZIP-Archive-Filename> -d <ACMATRIX_ROOT>` (e.g. `$ unzip AccessMatrix-5.3.2.3207-GA.zip -d /opt/acmatrix`).

4. After extracting all the files, go to the AccessMatrix Apache Tomcat's binary directory, i.e. */opt/acmatrix/tomcat/bin/*.
5. Issue the command `$ chmod +x *.sh` to change the file permissions for **startup.sh** to be executable.
6. To ensure that the installation is proper, execute the command `$ ./version.sh` or `$ bash version.sh`.
   If the command can be executed successfully, the system should return some Apache Tomcat related arguments and settings.

   Note: The Administrator can now proceed to operate on the AccessMatrix system.

Create AccessMatrix Database

Upon successful installation, if the administrator has not created the AccessMatrix security registry before or there is a schema change in the existing AccessMatrix security registry, the administrator may proceed to create/update the AccessMatrix security registry accordingly.

Note: Starting from 5.6.4-GA onward, an additional database script to create a view of audit tables is to be executed. However, an error will occur if SQL Server Management Studio is used to run the script. To fix this problem, there is a need to update the script by adding a GO statement before executing the create view script in SQL Server Management Studio.
Example:

PL/SQL

```
GO
create view am_vw_audit_b_a as select * from am_audit_b
union all select * from am_audit_a;
```

Create Security Registry

The administrator is advised to liaise with the Database Administrator (DBA) to create a database instance dedicated for hosting the security registry on the desired database server.

Important Note: AccessMatrix requires the database's Data/Character Set (a.k.a. "Server Collation") to be set to "Case-Sensitive". Failing to do so will require an administrator to recreate the database with the above setting again. After the database instance has been created, the administrator is required to run the AccessMatrix database create script

	manually to create the security registry. The database script is located in the *<ACMATRIX_ROOT>\dbscripts* folder.

Update Security Registry

To update the existing AccessMatrix security registry that has been set up by the previous AccessMatrix installation due to a change in the security registry schema in the latest release, the administrator is advised to liaise with the Database Administrator (DBA) to perform such action.

The administrator is required to run the AccessMatrix database update script manually to update the security registry. The database script is located at *<ACMATRIX_ROOT>/dbscripts/* folder.

Once the above is completed, proceed to configure the AccessMatrix Dispersed system to integrate with the desired database server that hosts the AccessMatrix security registry as described in the following sections.

Additional Privileges for Housekeeping

Periodically, AccessMatrix backend server (according to the configuration of housekeeping) archives audit logs (to be housekept later).

AccessMatrix housekeeping uses SQL Truncate to clean up the archived audit logs. Therefore, DB user which AccessMatrix uses to connect to DB (refer to Configure DataSource Connection to AM5), should have enough privileges to run the SQL truncate command.

In general, the DB user must be granted privileges for SELECT, INSERT, UPDATE and DELETE. However, for housekeeping (that uses SQL truncate), the DB user needs additional privileges.

<table>
<tr>
<td>Oracle</td>
<td>
For Oracle database, two stored procedures are being used to execute the SQL truncate command on audit tables. Ensure that the schema owner has CREATE ANY PROCEDURE privilege so they can create the two stored procedures in AccessMatrix script.

If DB user (which AccessMatrix uses to connect) does not own the schema (or if the schema is owned by the system administrator), then EXECUTE privilege shall be granted by the system administrator to the DB user. For example:
<table>
<tr><td>PL/SQL</td></tr>
<tr><td><pre>GRANT EXECUTE ON [schema].sp_truncate_am_audit TO [username]</pre></td></tr>
</table>
</td>
</tr>
</table>

```
GRANT EXECUTE ON [schema].sp_truncate_am_audit_attr TO
[username]
```

Note: If the DB user owns the schema, then EXECUTE privilege is not needed.

Run the following scripts if the stored procedures sp_truncate_am_audit and sp_truncate_am_audit_attr belong to the schema of other user.

PL/SQL

```
CREATE OR REPLACE SYNONYM
[username].sp_truncate_am_audit FOR
[schema].sp_truncate_am_audit
CREATE OR REPLACE SYNONYM
[username].sp_truncate_am_audit_attr FOR
[schema].sp_truncate_am_audit _attr
```

MSSQL

For MSSQL database, there are two alternatives:

- If DB user does not own the schema (or if the schema is owned by the system administrator), then ALTER privilege on the following tables shall be granted to the DB user.

  PL/SQL

  ```
  GRANT ALTER ON dbo.am_audit_a TO [username];
  GRANT ALTER ON dbo.am_audit_b TO [username];
  GRANT ALTER ON dbo.am_audit_c TO [username];
  GRANT ALTER ON dbo.am_audit_attr_a TO [username];
  GRANT ALTER ON dbo.am_audit_attr_b TO [username];
  GRANT ALTER ON dbo.am_audit_attr_c TO [username];
  ```

- If the DB user owns the schema, then additional privilege (ALTER) is not needed. To allow table creation in a user's own schema, the system administrator must first grant CREATE TABLE and CREATE SCHEMA privileges to the DB user. The system administrator shall modify the AccessMatrix DB script by adding schema name [schema] in front of the created DB objects. Get the **sqlserver-create.sql** file in the *<ACMATRIX_ROOT>/dbscripts/* folder, then modify it like in this example, and run the script as the DB user:

  PL/SQL

  ```
  create schema [schema];

  create table [schema].LOG_COMMAND (
         COMMANDID number(10,0) not null,
         COMMAND varchar2(500 char),
         BEGIN_TIME timestamp not null,
         SESSIONID varchar2(80 char),
         primary key (COMMANDID)
  ```

```
    );
 create table [schema].am_audit_attr_a (
        aa_server_id number(10,0) not null,
        aa_id number(19,0) not null,
        aa_name varchar2(64 char) not null,
        aa_part number(10,0) not null,
        aa_name_upper varchar2(64 char) not null,
        aa_datatype varchar2(16 char) not null,
        aa_value varchar2(3072 char),
        primary key (aa_server_id, aa_id, aa_name,
aa_part)
    );

 create view [schema].am_vw_audit_b_a as select * from
[schema].am_audit_b union all select * from
[schema].am_audit_a;
```

MYSQL

MYSQL does not have the schema ownership feature; the DB user shall be granted the least privilege DROP to truncate the audit tables.

MySQL

```
GRANT DROP ON amdb.am_audit_attr_a TO [username];
GRANT DROP ON amdb.am_audit_attr_b TO [username];
GRANT DROP ON amdb.am_audit_attr_c TO [username];

GRANT DROP ON amdb.am_audit_a TO [username];
GRANT DROP ON amdb.am_audit_b TO [username];
GRANT DROP ON amdb.am_audit_c TO [username];
```

Configure AccessMatrix Database Connection

The AccessMatrix server makes use of a system properties file to retrieve information on the Java Naming and Directory Interface (JNDI)-based DataSource connection properties. This is necessary for AccessMatrix to connect to a database residing in a dispersed database server.

Typically, the administrator is not required to configure the DataSource connection properties manually. However, there are cases where it may be necessary to do so, such as when the web server is not running on the bundled Tomcat web server (e.g. JBoss Web Server, etc.).

The AccessMatrix server will automatically detect the dialect to be used based on the DataSource name specified in the system properties file, provided the security registry is running on a supported Relational Database Management System (RDBMS).

If necessary, perform the following steps to configure the DataSource connection properties:

1. Navigate to the directory *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/classes/*.
2. Open the **amsystem.properties** file.
3. Search for **# DataSource** to locate the domain store-related properties (i.e. the DataSource connection properties for the desired database server):

Properties (.properties files)

```
# Datasource
# for Tomcat, please use the full datasource name (e.g.
java:/comp/env/jdbc/DSamdb)
# for Weblogic, please use only the short datasource name (e.g.
DSamdb)
# for other web apps, please use the relative datasource name (e.g.
jdbc/DSamdb)
am.domain.store.datasource=java:/comp/env/jdbc/DSamdb

# do not set or leave it empty for automatic detection of dialect
#am.domain.store.dialect=org.hibernate.dialect.DerbyDialect
# activate this line if MySQL version is 5.7 or 8.0.18
#am.domain.store.dialect=com.isprint.am.hibernate.AmMySQL57InnoDBDi
alect
```

4. Update the default values of the DataSource connection properties if needed:
    - Do not change the value for the DataSource name unless necessary.
    - If the AccessMatrix server fails to auto-detect the dialect upon startup, specify the dialect in the properties file by manually uncommenting and editing the dialect property.
    - Replace the value for the Hibernate SQL dialect class name with the relevant Hibernate SQL dialect class name of the underlying AccessMatrix database.
5. Save the changes made.

Configure Encryption Keys Expiry in Memory

There are two types of encryption keys in the AccessMatrix server. They are mainly:

- Keys stored in the Hardware Security Module (HSM), cloud or vault, where AccessMatrix server will refer to these keys by its label or index.
- Keys that are stored in AccessMatrix server, i.e. the keys of the System Key Provider that are secured or wrapped by Domain Master Key (DMK) or by HSM keys.

To reduce surface attacks from having clear (unwrapped) keys in memory, the administrator can set expiry of those keys. To do this,

1. Navigate to the directory *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/classes/*.
2. Open the **amsystem.properties** file.
3. Search for:

```
Properties (.properties files)

#AM crypto keys expiry, in seconds. If the value is not
set or less equal than 0, means no expiry.
#am.crypto.keys.expiry
```

4. Uncomment am.crypto.keys.expiry and set the expiry time in seconds. For example, you may set it to 300 seconds as shown below, or any other required value:

```
Properties (.properties files)

#AM crypto keys expiry, in seconds. If the value is not
set or less equal than 0, means no expiry.
am.crypto.keys.expiry=300
```

5. Save the changes made.

Configure Web Application Descriptor File

The administrator is required to set up the AccessMatrix web application descriptor file to enable the bundled Apache Tomcat Web server to connect to AccessMatrix database resides in a dispersed database server via a datasource connection.

To fulfil the above-mentioned task, perform the following steps:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\conf\Catalina\localhost*.
2. Open the **am5.xml** file.
3. Under the /am5 context path and the jdbc/DSamdb resource name, modify the datasource connection properties accordingly to cater for the desired database server.

   The following shows an example of defining the above-mentioned datasource connection properties to connect to MySQL database:

```
XML

<Resource name="jdbc/DSamdb" auth="Container" type="javax.sql.DataSource"
    description="AM5 DB"
    factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory"
```

```
initialSize="10" maxWaitMillis="10000" maxTotal="80" maxIdle="20"
minIdle="10"
 validationQuery="select 1"
    username="changeToDbUserId" password="changeToDBPassword"
    driverClassName="com.mysql.jdbc.Driver"

url="jdbc:mysql://10.1.11.85:3306/amdb?useUnicode=true&amp;characterEncod
ing=utf8"
/>
```

Important Notes:

- Ensure that the database's username and password are set correctly.
  Note: The Encryptor Utility may be used to encrypt the password.
- The value for JDBC driver class name is to be replaced with the relevant JDBC driver class name for the underlying AccessMatrix database.
- Ensure that the JDBC URL is set correctly to point to the underlying AccessMatrix database. If AccessMatrix database is to be run on MS SQL Server, the JDBC URL must be set to use these options: "selectMethod=direct; sendStringParametersAsUnicode=false".
  For example:
  jdbc:sqlserver://10.1.11.85:1433;database=amdb;selectMethod=direct;sendStringParametersAsUnicode=false.
  "selectMethod=cursor" should be used only if the AccessMatrix server instance is solely meant for dedicated reporting use. If the cursor method is selected, the SQL server's performance profiler produces a very unsearchable or unreadable trace that is very hard for identifying, which causes SQL statement to be slow in execution, especially during troubleshooting.
- If the database user's password has expired and needs to be updated, then the password provided in the datasource configuration should be updated accordingly.

4. Save the changes made.

Configure Admin Console Properties

The administrator is allowed to configure the default settings and advanced features of the Admin Console. Default settings such as default language or default authentication realm can be configured. Advanced features enable user selection of an authentication scheme used to log in to AccessMatrix Server.

These Admin Console properties are provided in following properties file:

<Web_Application_Server's_AxMx_Deployment_Folder>/WEB-INF/classes/wpe_config.properties.

Refer to AccessMatrix Admin Console Properties for details.

Copy JDBC Driver

To copy the JDBC driver:

1. Get the relevant JDBC driver classes ready.
2. Copy the JDBC driver to the directory *<ACMATRIX_ROOT>/tomcat/lib/*.
3. Restart the AccessMatrix Server.

Configure AM Log Housekeeping Properties File

The AM log housekeeping behavior can be configured via the **amlog4j2.properties** file at *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/classes/*. When deploying AccessMatrix, it is important to consider which housekeeping configurations will best suit your requirements.

Below are several key properties within the file and their descriptions:

Property Name	Description	Remarks
appender.LOG_ROOT.policies.time.interval	The number of days between each log rollover (e.g. "1").	The value is in days by default.
appender.LOG_ROOT.policies.size.size	The total size a log file can reach before it is rolled over (e.g. "500MB").	The unit of the value should be specified as well (e.g. KB, MB, GB).
appender.LOG_ROOT.strategy.fileIndex	The number of log files that can be stored before the next rollover. The following values can be configured: • "nomax" - This value means that there is no maximum	

Property Name	Description	Remarks
	number of log files that can be stored before the next rollover. • "max=A" - The value A represents the maximum number of log files that can be stored (e.g. "5"). When the counter exceeds A, older files will be deleted on subsequent rollovers. The default value is 7.	
appender.LOG_ROOT.strategy.delete.ifLastModified.age	The duration any log file can exist before it is deleted (e.g. "P5D").	The string entered here should begin with "P", followed by any of the following suffixes: - "D" for days. - "H" for hours. - "M" for

Property Name	Description	Remarks
		months - "S" for seconds For example, P5D indicates a duration of 5 days. The characters can be entered in lower or uppercase.
appender.LOG_ROOT.strategy.delete.ifAccumulatedFileCount.exceeds	The total number of log files that can be accumulated before older log files are deleted (e.g. "20").	For example, if the value entered is 20, the oldest log file will be deleted when the file count reaches 21.

The following is an example of the default configurations for the AccessMatrix standard log:

```
### AM STANDARD LOG ###
appender.LOG_ROOT.type = RollingFile
appender.LOG_ROOT.name = LOG_ROOT
appender.LOG_ROOT.fileName = logs/${hostName}_am.log
appender.LOG_ROOT.filePattern = logs/${hostName}_am.log.%d{yyyy-MM-dd}_%i.gz
# Uncomment line below to change compression from gz to zip
#appender.LOG_ROOT.filePattern = logs/${hostName}_am.log.%d{yyyy-MM-dd}_%i.zip
appender.LOG_ROOT.layout.type = PatternLayout
appender.LOG_ROOT.layout.charset = UTF-8
appender.LOG_ROOT.layout.pattern = %d{yyyy-MM-dd HH:mm:ss.SSS} [%t] %-5p - (%F:%L) %m%n
appender.LOG_ROOT.policies.type = Policies
appender.LOG_ROOT.policies.time.type = TimeBasedTriggeringPolicy
appender.LOG_ROOT.policies.time.interval = 1
appender.LOG_ROOT.policies.time.modulate = true
appender.LOG_ROOT.policies.size.type = SizeBasedTriggeringPolicy
appender.LOG_ROOT.policies.size.size=5120MB
```

```
appender.LOG_ROOT.strategy.type = DefaultRolloverStrategy
appender.LOG_ROOT.strategy.fileIndex = nomax
appender.LOG_ROOT.strategy.delete.type = Delete
appender.LOG_ROOT.strategy.delete.basePath = logs
#appender.LOG_ROOT.strategy.delete.ifLastModified.type = IfLastModified
#appender.LOG_ROOT.strategy.delete.ifLastModified.age = P2d
appender.LOG_ROOT.strategy.delete.ifFileName.type = IfFileName
appender.LOG_ROOT.strategy.delete.ifFileName.glob = *am.log.*
appender.LOG_ROOT.strategy.delete.ifAccumulatedFileCount.type = IfAccumulatedFileCount
appender.LOG_ROOT.strategy.delete.ifAccumulatedFileCount.exceeds = 180
```

As detailed in the above example, the default housekeeping configuration for AccessMatrix logs is as follows:

- Log rollover happens daily.
- Log files exceeding 5120MB are rolled over.
- There is no limit to the number of log files that can be stored between rollovers.
- Each log file is only kept for 2 days.

  > **Note**: This is disabled by default; uncomment the relevant code lines to enable this configuration.

- A total of 180 log files can be kept.

Refer to the Logging subsection of the *Common Administration Guide* for more details on log files.

Startup

This section describes the steps to startup AccessMatrix System on respective platforms.

Starting AccessMatrix Server on Windows	Under typical installation on Windows, AccessMatrix Server is started automatically as a service upon computer startup. If it has not started, the administrator may start it using one of the following ways. **Starting AccessMatrix Server in Services Panel** The administrator may start the AccessMatrix Server from the Windows Service panel. **Starting AccessMatrix Server in Console Mode** Alternatively, administrator may start the AccessMatrix Server from the Windows Start menu, i.e.: **Start > Programs > AccessMatrix > AccessMatrix Server > Operations > Start AccessMatrix Server (Console)**.
Starting AccessMatrix Server on Unix	If AccessMatrix Server has been registered for auto-startup on Unix, it will be started automatically as a background process upon computer startup. If it has not started, the administrator may start it as followed.

Starting AccessMatrix Server

1. Log in as the "acmatrix" account.
2. Su to the "root" account by entering the command `$ su`.
3. Go to AccessMatrix Apache Tomcat's binary directory, i.e. *<ACMATRIX_ROOT>/tomcat/bin/*.
4. Start AccessMatrix Server with the command `# ./startup.sh`.
5. Verify to ensure that the server has been started successfully by entering command `# ps -ef | grep /opt/acmatrix/tomcat`, where the system should return the running process of AccessMatrix Server.
6. Exit the "root" user session by entering the command `# exit`.

Registering/Unregistering AccessMatrix Server for Auto-Start in Unix

In Unix platform, upon successful installation of AccessMatrix system, the administrator may opt to perform the following functions to register/un-register AccessMatrix Server process as initial entry, so that AccessMatrix Server will be started automatically upon startup of the computer/operating system.

Auto Startup

1. Log in as the "root" account.
2. Create AccessMatrix auto-startup script (i.e. at */etc/init.d/axmx*) to allow AccessMatrix Server to start and run in the background upon computer startup or OS boot-up.
   The content of the script is shown below:

   Bash (Unix Shell)

   ```
   # chkconfig: 2345 56 80
   # description: AccessMatrix Server
   JAVA_HOME=<JAVA_Installation_Directory>
   export JAVA_HOME
   cd /opt/acmatrix/tomcat/bin
   nohup ./startup.sh
   ```

3. Give "execute" permission to the script by entering the command `# chmod +x axmx`.
4. Initialize the script by enter the command `# chkconfig --level 2345 axmx`.
5. Reboot the system so that the new settings become effective.

Unregister Auto Startup

1. Log in as the "root" account.
2. Delete the AccessMatrix auto-startup script (i.e. */etc/init.d/axmx*).
3. Reboot the system for new settings to take effect.

3.3. AccessMatrix Server as Docker Container

This section presents the sample steps for deploying the AccessMatrix server as a Docker container. The administrator is required to make changes to the **amsystem.properties** file to allow the containerization of AccessMatrix Server.

> **Important Note**: AccessMatrix server only supports Linux Containers, and does not support Windows Containers.

Mandatory Setting

Perform the following steps:

1. **Extract amsystem.properties** file from the CD image *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/classes/*.
2. Open the **amsystem.properties** file.
3. Modify and enable the following:

   Properties (.properties files)

   ```
   am.container.enabled=true
   ```

4. Place the edited **amsystem.properties** file in the same directory as the Dockerfile.

Optional Setting

If you need to use another Datasource like MySQL or SQLServer, you can configure the **am5.xml** file in a similar manner to the Mandatory Setting above.

Database Connection

1. Extract **am5.xml** file from the CD image *<ACMATRIX_ROOT>/tomcat/conf/Catalina/localhost/*.
2. Configure it according to Configure Web Application Descriptor File.
3. Place the JDBC driver that is being configured into the same directory as the Dockerfile.
4. Add the following lines to the Dockerfile:

   Code

   ```
   COPY ["am5.xml",
   "/opt/am/am/tomcat/conf/Catalina/localhost/"]
   COPY ["(DriverName).jar", "/opt/am/am/tomcat/lib/"]
   ```

Dockerfile Sample

Obtain a correct AccessMatrix package from the CD image - for example, **5.6.4.6409-GA-tomcat-only.tar.gz**. Place the AccessMatrix package in a folder called "repo" within the same directory as the Dockerfile.

Dockerfile

Code

```
FROM adoptopenjdk/openjdk11:slim
ENV AM_VERSION "5.6.4.6409-GA"
RUN mkdir -p /opt/am/${AM_VERSION}
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}
RUN groupadd am \
    && useradd -r -g am -s /bin/bash am \
    && chown -R am:am /opt/am/${AM_VERSION} \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh
\
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:am /opt/am/am
COPY ["amsystem.properties",
"/opt/am/am/tomcat/webapps/am5/WEB-INF/classes/"]
EXPOSE 8080 8443
USER am
CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
```

Build a Docker image for AccessMatrix Server

To build a Docker image using the "Dockerfile", run the command `docker build -t accessMatrix .`.

Logging

> **Note**: The default log files can be found in the folder */opt/am/am/tomcat/logs/*.

The log files can be externalized into an external volume during runtime. For example, use the command `docker container run -d --rm --mount source=[volume],target=/opt/am/am/tomcat/logs -p 8383:8080 [image]`.

3.4. AccessMatrix Server to Redis Server Connection

Redis is an external cache server that is used for cloud deployments where multicast is not available for EhCache RMI replication. In general, the selected cache provider (EhCache/Redis) will manage the cache storage.

There are several features available when Redis is selected as the cache provider:

1. Replication is no longer required for transient data as it will be cached in the Redis server.
2. Redis Pub/Sub can be configured for replication, even when data stored in the database is cached using EhCache.

> **Note:** For more information, please refer to Cache Configurations.

Configure Cache Provider

The administrator is required to change the **amsystem.properties** file to configure the cache provider.

1. Extract the **amsystem.properties** file from the CD image *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/classes/*.
2. Open the **amsystem.properties** file.
3. Modify and enable the following to use Redis as the cache provider:

Properties (.properties files)

```
#choose redis or ehcache, if not set, default is
ehcache
am.cache.vendor=redis
am.redis.host=127.0.0.1
am.redis.port=6379
#optional, don't set if no password required
am.redis.password=xxxx
#optional, default is false
am.redis.ssl.enabled=true
#optional, default is 8
am.redis.pool.maxTotal=8
#optional, default is 8
am.redis.pool.maxIdle=8
#optional, default is 3000
am.redis.pool.maxWaitMillis=3000
```

4. Save the edited **amsystem.properties** file in the same directory.

Mutual Authentication in AccessMatrix Server to Redis Server

AccessMatrix Server is connected to the Redis server by using an SSL certificate. To enable mutual authentication for Redis in AccessMatrix Server to Redis Server.

1. Set the following in the **amsystem.properties** file.

Properties (.properties files)

```
#optional parameters for setting up redis ssl
connection:
am.redis.ssl.keyStore
am.redis.ssl.keyStorePassword
am.redis.ssl.keyStoreAlias
am.redis.ssl.keyStoreKeyPassword
```

Note: The settings will only be effective if am.redis.ssl.enabled=true.

3.5. AccessMatrix Server with CLP for SSO to Admin Console

The AccessMatrix Admin Console supports Single Sign-On (SSO) using OpenID Connect (OIDC) as a login method for users. The AccessMatrix Common Login Page (CLP) serves as the OIDC OpenID Provider (OP), handling the user authentication.

This process involves the following AccessMatrix components:

- **AccessMatrix Server**
    - The server that the Admin Console is running on. The Admin Console redirects the user to CLP for login.
    - Serves as the OIDC Relying Party (RP) front end.
- **AccessMatrix Common Login Page**
    - The login page that the user will be redirected to for login. Handles user authentication.
    - Serves as the OIDC OP.
- **AccessMatrix OIDC Relying Party Servlet**
    - The servlet that handles OIDC communication with the OP.
    - Serves as the OIDC RP back end.

To use this feature, all three components must be first be deployed and configured. The instructions below assume that the AccessMatrix server has already been deployed.

Deploy Common Login Page

Refer to *AccessMatrix Common Login Page Deployment Guide* for instructions on deploying CLP and configuring the desired authentication methods.

Deploy OIDC RP Servlet

Refer to *UAM OIDC Relying Party Implementation Guide - OIDC Relying Party Servlet Implementation* for instructions on deploying the OIDC RP Servlet for authentication.

Configure AccessMatrix as OIDC RP

For AccessMatrix to act as the RP together with the RP servlet, you are required to perform the following configurations:

Configure Lookup Module

1. Navigate to **Administration Dashboard > Configuration > Authentication** and click **User Lookup Module** on the left pane.
2. Click the **Create** button and select "OIDC Relying Party Lookup Module" implementation from the **Choose User Lookup Module Implementation** page.
3. Click **Next**.
4. Select "Yes" under the **PKCE Flow is Used** attribute if the Authorization Code flow with PKCE is used. "No" is selected by default.

   > **Note:** Using the PKCE extension is a requirement for all new applications, based on the latest OAuth 2.1 specification.

5. Under the **OIDC RP Servlet** tab, enter the URL pointing to the AccessMatrix Admin Console for **Landing Page URL**. For example, if using the default URL, specify "http://localhost:8080/am5/wpe/WebPolicyEditor.jsp".
6. Provide the necessary values for the remaining lookup module attributes.
7. Save the changes.

> **Note:** Refer to *UAM OIDC Relying Party Implementation Guide - Configure Lookup Module* for detailed information on the attributes in this lookup module, as well as other additional configurations.

Configure Login Module

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.
2. Click **Create** and select the "Social Network Login" implementation from the **Choose Login Module Implementation** page.
3. Enter a **Login Module Id** (e.g. "SocNetLM") and a **Login Module Name** (e.g. "Social Network Login Module").
4. Select the **Login Policy** and the **Authentication Level** values, if necessary.
5. Click **Save**. A new login module is created.

Configure Authentication Realm

1. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm**.
2. Click **Create**.
3. Enter an **Authentication Realm Id** (e.g. "RPRealm") and an **Authentication Realm Name** (e.g. "OIDC RP Realm").
4. Specify the **User Lookup Module** value that will be used to validate user identity, e.g. "OIDC RP Lookup Module (OIDCRPLM)".
5. Specify the **First(1st) Login Module** that will be used for user authentication, e.g. "Social Network Login Module (SocNetLM)".
6. Click **Save**. A new authentication realm is created.

Assign Authentication Realm to User

If the user has not yet been created, navigate to **Operation Dashboard > User & Segment > User** and click **Create** on the right pane. Specify the **User Id** (e.g. "testuser") and **User Name** (e.g. "testuser") before proceeding with the remaining steps below.

Alternatively, select an existing user and click **Edit** in the **View User** page.

Once the above is done, do the following:

1. Under the **Login Accounts** tab, click the **Add button.** The **Choose Authentication Realm** dialog box is displayed.
2. Select the authentication realm created in previous section [e.g. OIDC RP Realm (RPRealm)] and click **OK**. Both authentication realm and the login module(s) associated with it are assigned to the user.
3. Click **Save**.

Configure AccessMatrix index.html File

Next, update the AccessMatrix **index.html** so that the user is redirected to CLP for login upon entering the AccessMatrix web app:

1. Navigate to *<AM_ROOT>\tomcat\webapps\am5* and open the **index.html** file.

2. Under the META tag, change the url value of the CONTENT attribute to "<AM_URL>/clp/rp/init/<realmId>", wherein:
    - <AM_URL> refers to the AccessMatrix URL (e.g. "http://localhost:8080").
    - <realmId> refers to the authentication realm Id of the previously configured authentication realm (e.g. "RPRealm") that contains the OIDC RP lookup module.

    For example:

    HTML

    ```
    <HTML>
    <HEAD>
    <META http-equiv="refresh" CONTENT="1;
    url=http://localhost:8080/clp/rp/init/RPRealm">
    </HEAD>
    </HTML>
    ```

3. Save the changes.

Configure Session Token Cookie Settings

To enable OIDC SSO with CLP, the session token cookie name of the AccessMatrix server must match that of the OIDC RP Servlet. Additionally, it is strongly recommended that you change the default name to a suitable, unique name. This is to prevent a situation where the same name is shared among various AccessMatrix components, which may cause complications.

The cookie names can be modified in the AccessMatrix server and OIDC RP Servlet's respective configuration files. The cookie name specified in the AccessMatrix server configuration file should correspond to the cookie name referenced in the OIDC RP Servlet configuration file.

Update Cookie Name in AccessMatrix Server Config File

1. Navigate to *<AM_ROOT>\tomcat\webapps\am5\WEB-INF\classes* and open the **wpe_config.properties** file.
2. Search for the authentication.wsasid.cookie.name setting:

    Properties (.properties files)

    ```
    #AM-11748 session token cookie name, default value is
    WSASID
    authentication.wsasid.cookie.name = WSASID
    ```

3. Replace the default value with an appropriate name (e.g. "AM5RPCookie"):

Properties (.properties files)

```
#AM-11748 session token cookie name, default value is
WSASID
authentication.wsasid.cookie.name = AM5RPCookie
```

4. Save the changes.

Update Cookie Name and Cookie Path in OIDC RP Servlet Config File

1. Navigate to *<AM_ROOT>\tomcat\webapps\oauth-rp\WEB-INF* and open the **web.xml** file.

> **Note:** The actual location of the *oauth-rp* folder may vary depending on where you chose to deploy it. Refer back to the deployment instructions detailed in Deploy OIDC RP Servlet if needed.

2. Search for the sessionTokenCookieName element:

XML

```
<context-param>
  <param-name>sessionTokenCookieName</param-name>
  <param-value>WSASID</param-value>  <!-- cookie name,
default value is WSASID -->
</context-param>
```

3. Replace the default value with the same name specified in the AccessMatrix **wpe_config.properties** file (e.g. "AM5RPCookie"):

XML

```
<context-param>
  <param-name>sessionTokenCookieName</param-name>
  <param-value>AM5RPCookie</param-value>  <!-- cookie
name, default value is WSASID -->
</context-param>
```

4. Search for the cookiePath element:

XML

```
<context-param>
  <param-name>cookiePath</param-name>
  <param-value>/oauth-rp</param-value>
</context-param>
```

5. Replace the default value with an appropriate path where the cookie is accessible by the Admin Console.

XML

```
<context-param>
  <param-name>cookiePath</param-name>
  <param-value>/am5</param-value>
</context-param>
```

Note: To prevent cookies from causing unintended SSO/login to other modules (e.g. usoserver, usossf), you should avoid using "/" for the cookie path. For example, you may consider deploying the OIDC RP servlet (or CLP) **.war** files into the context path of AM5 and specify the AM5 context path (i.e. /am5) as the cookie path instead.

6. Review the rest of the cookie settings to prevent unintended third-party cookie access.

XML

```
    <context-param>
        <param-
name>sessionTokenCookieMaxAgeInSec</param-name>
        <param-value>-1</param-value>  <!-- cookie will
expire after browser is closed if value is -1 -->
    </context-param>

    <context-param>
        <!--Allow cookie to set in this domain. If you
using Internet Explorer, this value is required to be
set. Default is empty-->
        <param-name>cookieDomain</param-name>
        <param-value></param-value>
    </context-param>
    <context-param>
        <param-name>cookieSameSite</param-name>
        <param-value>Strict</param-value> <!-- Default
is not set for SameSite -->
    </context-param>
```

7. Save the changes.

Additional Configurations for User Auto Provisioning

User auto-provisioning is the process where the AccessMatrix system creates a new user and automatically provisions the new user's data to the user store in the database. This is useful when user information is obtained from an external identity provider and it is necessary to create a corresponding user in the AccessMatrix system. Depending on your deployment, auto-provisioning may or may not be needed.

OP and RP with Shared Database

As detailed in the introduction to this page, the AccessMatrix server (and its components) acts as both the RP and OP for CLP SSO to the Admin Console. Typically, the same database is used for both the OP and RP when a single AccessMatrix server is used. If multiple AccessMatrix servers are used, the same database may be shared between them as well.

In such cases, user auto-provisioning is not required, as both the OP and RP shares the same database that already contains the user. It is thus sufficient to configure relevant user identity mapping attributes using the **OIDC Relying Party Lookup Module**.

OP and RP with Separate Databases

Although uncommon, there may be situations where the OP and RP and do not share the same database. For instance, separate AccessMatrix servers acting as the OP and RP may be using different databases. In such cases, user auto-provisioning can be used if:

- The users had not already been provisioned manually.
- There are no concerns with performing user auto-provisioning during runtime.

To use the advanced user auto-provisioning feature, perform the following:

1. Navigate to **Administration Dashboard > Configuration > Authentication > User Auto Provisioning Module**.
2. Click the **Create** button, select the "Default User Auto Provisioning Handler" implementation, and click **Next.**
3. Enter a **User Auto Provisioning Module Id** (e.g. "AutoProv") and a **User Auto Provisioning Module Name** (e.g. "OIDC RP User Auto Prov").
4. Specify the **User Store** that contains the AccessMatrix users.
5. Specify the **Mapping User Id** value that indicates the payload's claim/attribute name of an entity (i.e. the user). This value will be mapped to the AccessMatrix user's user Id.
    - Alternatively, specify "userIdentity". This is a special payload that will contain the value of a specific claim in the OIDC ID token of the user. The claim used depends on the configurations made to the attributes in the OIDC Relying Party lookup module under the **User Identity Mapping** header.

> **Note:** Refer to *UAM OIDC Relying Party Implementation Guide - User Auto Provisioning Example Use Case* for more details on this.

6. Specify the **Mapping User Name** value that indicates the payload's claim/attribute name of an entity (i.e. the user). This value will be mapped to the AccessMatrix user's username.
    - Alternatively, specify "userIdentity". This is a special payload that will contain the value of a specific claim in the OIDC ID token of the user. The claim used depends on the configurations made to the attributes in the OIDC Relying Party lookup module under the **User Identity Mapping** header.

    > **Note:** Refer to *UAM OIDC Relying Party Implementation Guide - User Auto Provisioning Example Use Case* for more details on this.

7. Under **Mapping User Attributes**, enter the required values for the mapping of the payload's claims to AccessMatrix (the OIDC RP).
8. Click **Save**. A new user auto provisioning module is created.

3.6. AccessMatrix Advanced Installations

AccessMatrix Distributed system is installed by deploying the AccessMatrix Web Archive on a supported Web application server and supported JRE. Bind the Metaspace especially if the server is used to host other important processes. Maximum heap size is at least 1G for 64-bit. Adjust accordingly in the production deployment depending on the features and load.

The deployment is platform-independent and the AccessMatrix Web Archive is generated by running the AccessMatrix Installer which can only be run on Windows.

To generate the AccessMatrix Web Archive, perform the following steps on a computer running Windows operating system:

1. Insert the AccessMatrix Installation CD into the CD-ROM drive or run the Setup executable file directly.
2. Click **Next** to proceed with the installation and the **License Agreement** screen will appear.
3. Accept the terms of the license agreement about AccessMatrix and click **Next** to continue.
4. Select the directory to which the AccessMatrix Web Archive will be generated.
    - All the necessary files will be installed in this directory. The default directory is *C:/Program Files/AccessMatrix/*.
    - Click **Browse** to change the default root directory, or **Next** to accept the default directory and continue.
5. Select the components to be installed and deselect the components not to be installed as described below:
    - **AccessMatrix Server (OPTIONAL)** inclusive of the core server i.e. AccessMatrix Server and Apache Tomcat Web application server.
    - **USO Trainer (OPTIONAL)** is an independent component that is used to train the application login and change password screens. It can be installed separately by itself on another computer.
    - **AccessMatrix Web Archive (MUST SELECT)** will generate the AccessMatrix Web Archive that encompasses AccessMatrix Server and AccessMatrix Web Applications such as USO Agent, Admin Console, etc.
      For the AccessMatrix Distributed system installation, the administrator must check the **AccessMatrix Web Archive** option. It is not required to check the **AccessMatrix Server**

option.

After finish selecting the desired options, click **Next** to continue where the administrator will be prompted to enter the datasource information.

6. Specify the datasource name as well as the datasource type.

> **Note:** The default value may be accepted for the datasource name. If a new value is specified for the datasource name, keep a note of the value as it will be used during the AccessMatrix Web Archive deployment onto the desired Web application server. The value also depends on the format recognized by the Web application server and is generally in the form of "jdbc/<AxMx_DS_Name>" or "<AxMx_DS_Name>" where AxMx_DS_Name refers to the AccessMatrix datasource name, e.g. "jdbc/DSamdb" or "DSamdb".
>
> The datasource type refers to the supported database system such as Apache Derby, MS SQL Server, MySQL, and Oracle that will host the AccessMatrix Security Registry.

7. After specifying the datasource name as well as the datasource type, click on **Next** to continue.
8. If applicable, specify the unique AccessMatrix Server ID, and then click on **Next** to continue.
9. Review the installation settings and then click **Next** to start copying files or **Back** to change any settings. After clicking **Next**, the Installer will start copying files to the designated installation directory.
10. Upon successful installation, a screen will appear to inform the administrator to execute the AccessMatrix database setup or update script where applicable.
11. Click **OK** to continue.
12. Click on **Finish** to exit the installation wizard.

Upon successful installation, i.e. having generated the AccessMatrix Web Archive, if the administrator has not created the AccessMatrix security registry before or there is a schema change in the existing AccessMatrix security registry, the administrator may proceed to Creating/Updating AccessMatrix Security Registry section to create/update the AccessMatrix security registry accordingly.

Download WAR File

AccessMatrix installation package CD comes with a WAR file (e.g. **AccessMatrix-x.x.xxxx-GA.war**). Copy the WAR file to the <*ACMATRIX_ROOT*>. The WAR file may be renamed as **am5.war** for easy reference.

Deploy AccessMatrix on Red Hat JBoss Web Server

This section presents the steps for deploying the AccessMatrix Web Archive on Red Hat JBoss Web Server (JWS) 6.0 running on Windows.

> **Note:** Ensure all necessary zip files (eg. **jws-application-servers-6.0.0.zip** & **jws-application-servers-6.0.0-<platform>-<architecture>.zip**) are downloaded, and extract the contents to the installation directory (eg. *jws-6.0/*).

Increase Java Virtual Machine (JVM) memory

The default JVM memory allocation is insufficient to run AccessMatrix. The allocation should be increased as follows:

1. Run *<JWS_HOME>\tomcat\bin\tomcat<version>w.exe*.
   Example: *C:\jws-6.0\tomcat\bin\tomcat9w.exe*.
2. If there is a UAC prompt, click **Yes** to allow.
3. Click the **Java** tab.
4. Increase **Maximum memory pool** to 1024 MB.
5. Increase **Initial memory pool** to 400 MB.
6. Click **OK** to save.

Configure DataSource Connection to AM5

Perform the following steps to configure the datasource connection to AM5 database:

1. Add the JDBC driver(s) provided by the vendor into *<JWS_HOME>\tomcat<version>\lib*.
   Example: *C:\jws-6.0\tomcat\lib*.
2. Add the **am-tomcat-tools.jar** file provided by i-Sprint into *<JWS_HOME>\tomcat<version>\lib*.
   Example: *C:\jws-6.0\tomcat\lib*.
3. Create the AM5 XML file, i.e. *<JWS_HOME>\tomcat<version>\conf\Catalina\localhost\am5.xml*.
   Example: *C:\jws-6.0\tomcat\conf\Catalina\localhost\am5.xml*.
4. Add one of the following:

Derby	XML

```
<?xml version="1.0" encoding="UTF-8"?>
<Context path="/am5">
    <Resource name="jdbc/DSamdb" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"

factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory"
initialSize="10" maxTotal="80"
    maxIdle="20" minIdle="10" maxWaitMillis="10000"
validationQuery="SELECT COUNT(*) FROM am_vars WHERE va_name IS
NULL"
    testOnBorrow="true" username="user1" password="password"
driverClassName="org.apache.derby.jdbc.EmbeddedDriver"
    url="jdbc:derby:amdb"
/>
</Context>
```

MySQL

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<Context path="/am5">
    <Resource name="jdbc/DSamdb" auth="Container" type="javax.sql.DataS
description="AM5 DB"
    factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory"
initialSize="10" maxWaitMillis="10000"
    maxTotal="80" maxIdle="20" minIdle="10" validationQuery="select 1"
username="changeToDbUserId"
    password="changeToDBPassword" driverClassName="com.mysql.jdbc.Drive

url="jdbc:mysql://10.1.11.85:3306/amdb?useUnicode=true&amp;characterEnc
/>
</Context>
```

Oracle

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<Context path="/am5">
    <Resource name="jdbc/DSamdb" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"

factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory"
initialSize="10" maxTotal="80"
    maxIdle="20" minIdle="10" maxWaitMillis="10000"
validationQuery="select 1 from dual"
    testOnBorrow="true" username="SYSTEM" password="password1"
    driverClassName="oracle.jdbc.driver.OracleDriver"
url="jdbc:oracle:thin:@127.0.0.1:1521:AMDB"
/>
</Context>
```

Microsoft SQL Server

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<Context path="/am5">
```

	<Resource name="jdbc/DSamdb" auth="Container" type="javax.sql.DataSource" description="AM5 DB" factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory" initialSize="10" maxWaitMillis="10000" maxTotal="80" maxIdle="20" minIdle="10" validationQuery="select 1" username="sa" password="password" driverClassName="com.microsoft.sqlserver.jdbc.SQLServerDriver" url="jdbc:sqlserver://192.168.1.205:1433;databaseName=amdb; sendStringParametersAsUnicode=false" /> </Context>
Dameng	XML <?xml version="1.0" encoding="UTF-8"?> <Context path="/am5"> <Resource name="jdbc/DSamdb" auth="Container" type="javax.sql.DataSource" description="AM5 DB" factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory" initialSize="10" maxTotal="80" maxIdle="20" minIdle="10" maxWaitMillis="10000" validationQuery="select 1 from dual" testOnBorrow="true" username="user2" password="password1" driverClassName="dm.jdbc.driver.DmDriver" url="jdbc:dm://192.168.50.103/DAMENG" /> </Context>

Note: The Encryptor Utility may be used to encrypt the password.

Deploy AccessMatrix Web Archive

1. Rename the AM5 WAR file (e.g. **AccessMatrix-<version>-GA.war**) as **am5.war**, for a simpler URL.
2. Copy **am5.war** into *<JWS_HOME>\tomcat<version>\webapps, e.g. C:\jws-6.0\tomcat\webapps*.
3. Start JWS.
4. The folder *<JWS_HOME>\tomcat<version>\webapps\am5* will be auto-created, e.g. *C:\jws-6.0\tomcat\webapps\am5*.

(OPTIONAL) Monitoring JWS Tomcat's Thread Count

AccessMatrix server has the functionality to monitor Tomcat's thread count. To enable this optional feature, perform the following configurations:

1. Log in to AccessMatrix Admin Console and navigate to **Administration Dashboard > System > Server Monitoring**.
2. Click **Edit** and input "http-apr-8080" as the value for the **Tomcat Connector Name** attribute.
3. Click **Save** and restart AccessMatrix server for the change to take effect.

Note: If there's a change in Tomcat's connector's protocol, i.e. the connector's interface or connector's port, the value for Tomcat Connector Name should be changed accordingly. To find the correct Tomcat Connector Name, check **tomcat.log** or **catalina.log** for the phrase Initializing ProtocolHandler.
E.g.:

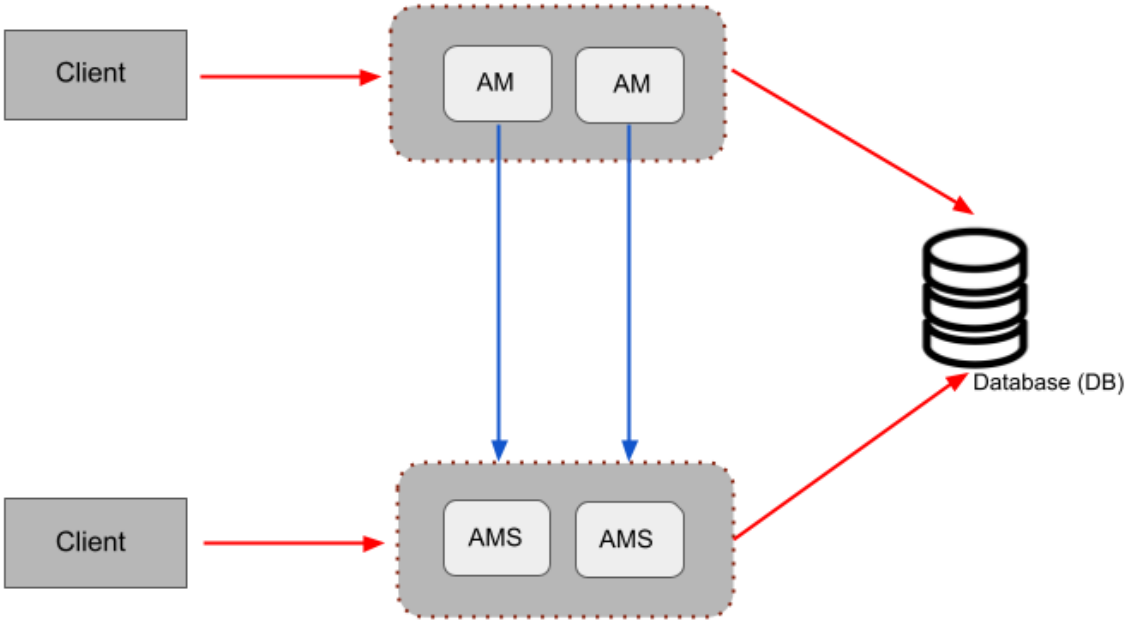
```
Initializing ProtocolHandler ["http-apr-8080"]
Initializing ProtocolHandler ["http-nio-8080"]
Initializing ProtocolHandler ["https-jsse-nio-8443"]
```

Deploy AccessMatrix Audit Microservices Separately

By default, you can install AccessMatrix manually or by running Window Installer as usual on-premise.

From version 5.7.0 onwards, auditing functions can be delegated to a separate Audit Microservice (AMS) to relieve the load on AccessMatrix and improve performance during authentication. To go for this option, Audit Microservices (AMS) must be deployed in a different Tomcat or app server.

AccessMatrix will write the audits to AMS by calling API. All historical reports that currently read from audit tables in AccessMatrix will be moved to AMS.



Setting up Audit Microservices

Audit Microservice (AMS) can be downloaded as a war file named **AccessMatrix-x.x.x.xxxx-xxx-am-audit.war**.

Unzip the war file and configure *WEB-INF/classes/ams.properties*.

ActiveMQ Configuration

1. By default, the activemq.url is commented out as AMS is not using ActiveMQ.
2. See the next section for the ActiveMQ configuration details.

Database Connection Configuration

Properties (.properties files)

```
#hibernate.dialect=org.hibernate.dialect.DerbyDialect
#hibernate.connection.datasource=java:/comp/env/jdbc/DSam-audit
hibernate.connection.driver_class=org.apache.derby.jdbc.ClientDriver
hibernate.connection.url=jdbc:derby://localhost:1527/amdb;create=false
hibernate.connection.username=user1
hibernate.connection.password=password
```

1. If using JNDI data source, uncomment and configure the hibernate.connection.datasource; comment out the rest of the properties.

2. Otherwise, comment out hibernate.connection.datasource and configure the rest of the connection properties.

> **Note:** Encryptor Utility may be used to encrypt the password.

Housekeeping Configuration

1. If AMS is deployed, then it will also take over the audit housekeeping tasks from AccessMatrix.
2. The default configuration should suffice for most deployments.

Properties (.properties files)

```
scheduler.names=ams-cluster,local
scheduler.ams-cluster.scheduler.instanceName=ams-cluster
scheduler.ams-
cluster.scheduler.instanceIdGenerator.class=com.isprint.am.microser
vice.audit.scheduler.AMSInstanceIdGenerator
scheduler.ams-cluster.threadPool.threadCount=5
scheduler.ams-cluster.scheduler.instanceId=AUTO
scheduler.ams-cluster.jobStore.isClustered=true
scheduler.ams-
cluster.jobStore.class=org.quartz.impl.jdbcjobstore.JobStoreTX
scheduler.local.scheduler.instanceName=local
scheduler.local.scheduler.instanceIdGenerator.class=com.isprint.am.
core.scheduler.AMInstanceIdGenerator
scheduler.local.scheduler.instanceId=AUTO
scheduler.local.threadPool.threadCount=5
scheduler.local.jobStore.class=org.quartz.simpl.RAMJobStore
```

Using ActiveMQ

There is one more option to improve the performance and availability of AMS, to use ActiveMQ to write audits asynchronously.

ActiveMQ must be running before starting up AMS.

Configure ams.properties

Properties (.properties files)

```
# - Active MQ configuration
# - uncomment url if using Active MQ
activemq.url=tcp://192.168.50.51:61617
# - if user authentication is required, uncomment
username, password (password can be encrypted with
EncryptorUtil)
activemq.username=
activemq.password=
```

1. Set the URL with the respective ActiveMQ protocol; if using SSL, ensure the required certificate and trust relationships are also set up.
2. If user authentication is required,
    a. Uncomment and set the username.
    b. Set the password.

> **Note:** Encryptor Utility may be used to encrypt the password.

Set AccessMatrix to connect to Audit Microservices

Audit Microservices must be evoked first before starting up AccessMatrix.

Finally, configure AccessMatrix to connect to Audit Microservices in **amsystem.properties**.

1. Set the URL and SSL client properties. Connection to AMS requires SSL with client authentication.

Properties (.properties files)

```
# == Audit Microservice (AMS) Configuration ==
# Uncomment the url to enable AM to use AMS
am.microservice.audit.url = https://testserver:8443/am-audit/rest
am.microservice.audit.ssl.client.keystore = ../conf/testagent.keystore
am.microservice.audit.ssl.client.keystore.passphrase = passphrase
```

> **Note:** Encryptor Utility may be used to encrypt the passphrase.

2. The default connection and timeout settings should suffice for most deployments. You may adjust them if you need to tune performance later.

Properties (.properties files)

```
# - if not specified below, defaults to 30
#am.microservice.audit.connection.timeoutSecs = 30
# - if not specified below, defaults to 30
#am.microservice.audit.request.timeoutSecs = 30
# - if not specified below, defaults to 10
#am.microservice.audit.connection.max = 10
# - if not specified below, defaults to 5 (recommended half of max connections)
#am.microservice.audit.connection.perRoute = 5
# - if not specified below, defaults to 30
#am.microservice.audit.socket.timeoutSecs = 30
```

3.7. OpenTelemetry

AccessMatrix is OpenTelemetry compatible for distributed tracing. OpenTelemetry is an open-source project that aids applications in emitting and collecting helpful data for troubleshooting purposes.

While AccessMatrix already provides data in the form of comprehensive logs and metrics, the use of automatic instrumentation as part of OpenTelemetry enables access to another form of data — traces.

> **Note:** Traces show the route that a request takes as it travels through the different applications or systems.

This additional information is useful for troubleshooting issues that arise in complex distributed systems, where requests often interact across different services and issues can be difficult to isolate. For example, trace IDs that are returned from AccessMatrix REST APIs allow developers to pinpoint the exact system where the issue occurs, even as it moves through multiple external systems.

Enable OpenTelemetry

> **Note:** Before proceeding, ensure that the version of OpenTelemetry currently in use coincides with the version included within AccessMatrix. By default, the **opentelemetry-javaagent.jar** file is located at *<ACMATRIX_ROOT>/tomcat/lib* and can be replaced if necessary.

To enable automatic instrumentation in AccessMatrix, perform the following:

Windows

1. Navigate to *<ACMATRIX_ROOT>/tomcat/bin* and open the **setenv.bat** file using a text editor.
2. Uncomment the following command line:

```
::set JAVA_OPTS=%JAVA_OPTS% -
javaagent:%CATALINA_HOME%\lib\opentelemetry-
javaagent.jar -
Dotel.instrumentation.quartz.enabled=false -
Dotel.resource.attributes=service.name=accessmatrix -
Dotel.exporter.otlp.endpoint=http://localhost:4318/
```

3. Modify the OpenTelemetry endpoint URL. For example:

```
-Dotel.exporter.otlp.endpoint=http://localhost:1234/
```

	4. (Optional) Modify the file path to the **opentelemetry-javaagent.jar** file, if necessary. For example:
	Code
	`-javaagent:C:\otel\opentelemetry-javaagent.jar`
	5. (Optional) Specify the term that will be used to identify AccessMatrix data when querying the OpenTelemetry backend, if necessary. For example:
	Code
	`-Dotel.resource.attributes=service.name=accessmatrix123`
	6. Save the changes.

Linux	1. Navigate to *<ACMATRIX_ROOT>/tomcat/bin* and open the **setenv.sh** file using a text editor.
	2. Uncomment the following command line:
	Code
	`#JAVA_OPTS="$JAVA_OPTS -javaagent:$CATALINA_HOME/lib/opentelemetry-javaagent.jar -Dotel.instrumentation.quartz.enabled=false -Dotel.resource.attributes=service.name=accessmatrix -Dotel.exporter.otlp.endpoint=http://localhost:4318/"`
	3. Modify the OpenTelemetry endpoint URL. For example:
	Code
	`-Dotel.exporter.otlp.endpoint=http://localhost:1234/`
	4. (Optional) Modify the file path to the **opentelemetry-javaagent.jar** file, if necessary. For example:
	Code
	`-javaagent: /otel/opentelemetry-javaagent.jar`
	5. (Optional) Specify the term that will be used to identify AccessMatrix data when querying the OpenTelemetry backend, if necessary. For example:
	Code
	`-Dotel.resource.attributes=service.name=accessmatrix123`
	6. Save the changes.

3.8. Special Considerations

This section describes special considerations the administrator may wish to take into account while deploying AccessMatrix.

Temporary Report Directory

When the AccessMatrix system receives a request to generate a report, the system will generate a report resulting in any of these report files: CSV, PDF, or HTML. The generated report file(s) will be stored temporarily in a reports folder that is automatically created when the AccessMatrix system generates a report for the first time. The reports folder will be created under this directory: *<ACMATRIX_ROOT>/work/*. Example for Tomcat: *E:/AM-Build/5.6.9.6911-GA/tomcat/work/reports/*.

Generally, the generated report file(s) in this directory will be removed after AccessMatrix has completely stream the report to the user. However, if for any reason the generated report is not be deleted (i.e. report generation is aborted halfway by the user or an unexpected error occurs), a daily report cleaner (ReportCleanerSchedJob) is scheduled to run at 4:00 AM every day.

Multiple AccessMatrix Servers

When there are multiple instances of AccessMatrix servers, the temporary report directory should be backed by a shared file system (e.g. PVC, NFS, CIFS). This is to allow the generation and download of reports to be served by different instances when the request comes through a load balancer.

Additionally, the following configuration must be made:

1. Navigate to *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/classes/* and open the **amsystem.properties** file.
2. Search for the #Shared temporary report folder configuration comment.
3. Uncomment #am.workdir = and configure the value to the path that the shared temporary report folder should be generated in. For example:

   Properties (.properties files)

   ```
   #Shared temporary report folder configuration
   am.workdir = X:/am/work
   ```

4. Save the changes.

The */report* folder will be generated in the location configured in the steps above.

Note: The above steps are mandatory when using multiple AccessMatrix servers. Failing to do so will result in errors such as the following:

```
2023-05-24 01:47:38.245 [http-nio2-8080-exec-3] INFO -
(ReportService.java:717) Report cache key not found, either it is expired or it
is an invalid key
2023-05-24 01:47:38.245 [http-nio2-8080-exec-3] ERROR -
(ReportGeneratorServlet.java:72) Exception while generating report-
EventsReport, msg=Report cannot be retrieved based on the report cache
key, File name: ReportGeneratorServlet.java, Class name:
com.isprint.am.gwt.core.server.service.ReportGeneratorServlet, Method
name: doGet, Line number: 98
```

3.9. Cache Configurations

AccessMatrix makes use of caches to speed up the performance of various components, especially those related to persistence. Ehcache is used as the cache component. Ehcache is a well known Java library that provides a caching mechanism. It is also used by Hibernate for Data Access Object (DAO) caching. This document explains how the AccessMatrix server uses Ehcache and how Ehcache can be configured.

Caches in AccessMatrix

- **Hibernate Cache**

  Hibernate uses Ehcache to cache the raw data of objects retrieved from the database. This cache is used when there are multiple queries to retrieve the same object. For example, it is used when a get user function is called multiple times for the same UUID.

> **Note**: Since version 5.6.4 of AccessMatrix, Hibernate's use of Ehcache has been disabled by default. This increases the reliability of modifying an object over multiple API requests. Such a scenario is common during the token activation process. Disabling this cache has shown only a negligible performance impact during stress testing. This is because the database server has its own caching system that works well with primary key-based queries (e.g. UUID-based queries).
>
> If you require Ehcache to be enabled, add the following line in the **amsystem.properties** file:
> am.domain.store.hibernate.cache.use_second_level_cache=true

- **Database-backed Object Cache** (CachedObjectRepository, ESSO cache, SAML Issuer cache, etc.)

  AccessMatrix has a cached object repository that caches most of the configuration and policy objects, login modules, lookup modules, groups, etc. Objects stored in this cache are considered complete. For example, a policy object contains all the configurations/attributes of its respective policy.

  - This cache is typically used by the business logic of AccessMatrix, such as its authentication engine, SAML service, etc. The cache tends to be more time-efficient than the Hibernate cache, as it is readily available within the AccessMatrix server memory.

- **Transient Data Cache**

  AccessMatrix modules can use a cache as primary storage. This means that the data resides only in the cache and not in the database. Such caches are used to store transient data with short lifespans. For example:

- **E2EE Session**

  AccessMatrix uses a cache to validate E2EE sessions. An E2EE session is a client-state session where session information is encrypted, integrity-checked, and stored client-side. Such sessions are one-time use only to avoid replay attacks and thus have short lifespans, typically under 10 minutes.

- **SMS-OTP MemoryTokenStore**

  AccessMatrix uses a cache to store SMS-OTP tokens. SMS-OTPs are generally short-lived, as they are generated and used within seconds/minutes.

The following tables provide an overview of the various implementations available for each cache, based on the AccessMatrix deployment environment:

AccessMatrix Caches for Non-cloud Environment				
Types of Caches	**Disabled**	**Ehcache RMI**	**Ehcache JMS**	**Redis**
Hibernate Cache	**Disabled by default**	Can be configured	Can be configured	Not available
DB-backed Object Cache	Cannot be disabled	**Used by default**	Can be configured	Can be configured
Transient Data Cache	Cannot be disabled	**Used by default**	Can be configured	Can be configured

AccessMatrix Caches for Cloud Environment			
Types of Caches	**Disabled**	**Ehcache JMS**	**Redis**
Hibernate Cache	**Disabled by default**	Not available	Not available
DB-backed Object Cache	Cannot be disabled	Can be configured	Can be configured
Transient Data Cache	Cannot be disabled	Not available	**Must be configured**

Refer to E. AccessMatrix Ehcache Configuration for the steps needed to change the preconfigured AccessMatrix caches.

Replication

Caches improve performance by having additional copies of data stored in memory. However, issues arise when cache entries are outdated. Thus, in a set-up with multiple AccessMatrix server instances, it is imperative to have cache replication.

Cache replication is not only used for sending values over to other servers for replication. For instance, in the AccessMatrix server, Ehcache replication is mostly used to invalidate cache entries in other AccessMatrix server instances so that new queries will go to the database. There are, however, specific caches where values need to be replicated, such as the E2EE session cache and SMS-OTP cache; in such instances, caches are used as primary storage instead of the database.

There are two considerations for cache replication: how to find other server instances and how information is replicated to other instances. AccessMatrix supports three methods for replication:

- **RMI (Java Remote Method Invocation)**
  RMI requires additional RMI ports to be opened. A multicast group IP address and port should be configured to enable the discovery of other instances.
- **JMS (Java Message Service)**
  JMS requires a compatible messaging server component. For JMS, all AccessMatrix server instances both publish replication events and subscribe to replication event publications. The cache discovery method is also based on the JMS subscription.
  This type of replication is meant for a Kubernetes deployment, or any cloud environment where server instances need to be horizontally scaled in a more flexible manner and multicast communication is unavailable. Only Amazon MQ and Azure Service Bus are officially supported as messaging servers.
- **Redis PubSub**
  Redis PubSub (Publish/Subscribe) is a messaging pattern supported by Redis that allows for decoupled communication between server instances. It requires a connection to a Redis server.

RMI Replication

Ehcache Peer Discovery

The standard/default way to configure Ehcache Peer Discovery is to configure the Ehcache configuration file. Ehcache has two modes of peer discovery.

- **Automatic/Multicast**

  Automatic uses multicast. That means we need to assign a specific multicast group IP address and port to be used by Ehcache. Ehcache will listen to this IP address and port and also will broadcast its existence to this IP address and port. This way, Ehcache can discover other instances. Ehcache will maintain and keep updating the list of peers based on the multicast group. It will know if any of the peers are still alive (other AccessMatrix servers are running) or not. When a peer is offline, it will be removed from the list, and the replication process will not replicate to the offline instance.
- **Manual**

  Manual requires the list of peers to be updated manually. No multicast IP address and port are used. There are two ways to configure a list of peers manually: Ehcache configuration file or programmatically. The list is static. It means if an AccessMatrix server instance is offline, then that instance is still not removed from the list of peers. This may slow the replication process as Ehcache will always try to replicate to the offline instance. Thus, it is recommended to use multicast.

AccessMatrix Ehcache Peer Discovery

Configuring Ehcache peer discovery via Ehcache configuration file is error-prone, especially for the Ehcache manual peer discovery. Thus, the AccessMatrix server has a unique configuration in **amsystem.properties** to assist the Ehcache peer discovery:

Properties (.properties files)

```
# == Ehcache ==
# am.ehcache.config_file
#   : Ehcache configuration file (default is am5-
ehcache.xml)
#
# To configure Ehcache replication/peer discovery:
# am.ehcache.peer_discovery
#   : Method to find peers
#     - db (default) = configure ehcache manual peer
discovery using servers information in database
#     - multicast    = configure ehcache automatic peer
discovery using multicast
#     - manual       = full manual, use the ehcache
configuration as it is, AccessMatrix will not assist in
setting up the replication
# am.ehcache.multicast_addr
#   : Multicast group address (required if
peer_discovery is set to multicast)
```

```
# am.ehcache.multicast_port
#   : Multicast group port (required if peer_discovery
is set to multicast)
# am.ehcache.multicast_ttl
#   : Determines how far the packets will propagate, 0
to 255
#     By convention, the restrictions are:
#       - 0   = the same host
#       - 1 (default) = the same subnet
#       - 32  = the same site
#       - 64  = the same region
#       - 128 = the same continent
#       - 255 = unrestricted
```

For this feature to work, the replication part of the Ehcache configuration (**am5-ehcache.xml**) needs to be left to its default value. In the AccessMatrix server, there are three possible configurations for peer discovery:

<table>
<tr><td>Multicast (recommended)</td><td>Properties (.properties files)

am.ehcache.peer_discovery=multicast
am.ehcache.multicast_addr=...
am.ehcache.multicast_port=...
am.ehcache.multicast_ttl=...

AccessMatrix will initialize Ehcache to use multicast and to listen to the multicast IP address and port as specified. Multicast Time-to-live (TTL) can be configured to control the propagation of multicast broadcast. Typically, the default value is good enough for AccessMatrix server instances running in the same network.</td></tr>
<tr><td>Database (DB) - (default)</td><td>Properties (.properties files)

am.ehcache.peer_discovery=db

DB means AccessMatrix will configure Ehcache peer discovery to manual and AccessMatrix will assist the configuration of the peer list. AccessMatrix uses its list of servers as configured in the Admin Console to find out the IP addresses of the other AccessMatrix servers. Then AccessMatrix will configure them as the peers.

If there is only one server in the server list, AccessMatrix will not enable Ehcache replication.</td></tr>
<tr><td>Manual/Ehcache file only</td><td>Properties (.properties files)

am.ehcache.peer_discovery=manual</td></tr>
</table>

	This "Manual" of AccessMatrix peer discovery is not to be mistaken with the "Manual" of Ehcache peer discovery. The more proper name for this is "Use Ehcache file without AccessMatrix assistance". With this setting, AccessMatrix will not try to configure the Ehcache peer discovery at all. Thus, it uses the peer discovery setting in the Ehcache configuration file as-is. This is useful if you need to configure peer discovery yourself in **am5-ehcache.xml** or for turning off replication for whatever reason.

Ehcache RMI Replication

The earlier section was discussing peer discovery. Now that peers are discovered or listed, this section explains the replication itself. Ehcache uses RMI to send the replication message. Thus, Ehcache needs to listen to two ports (replication port and remote object port). These ports can be configured in AccessMatrix server configuration via Admin Console. When AccessMatrix Ehcache Peer Discovery is configured to multicast or DB (am.ehcache.peer_discovery=multicast or am.ehcache.peer_discovery=db respectively), AccessMatrix will automatically configure the replication and remote object port based on what is configured in the Admin Console. These ports need to be opened in the firewall if there is a firewall between AccessMatrix server instances.

A restart of ALL servers is required if the cache replication port or remote object port is modified.

JMS Replication

JMS (Java Message Service) is an Application Program Interface (API) that supports the messaging communication between computers in a network. To implement JMS for Ehcache replication to AccessMatrix, the administrator or application developer has to configure **amsystem.properties** accordingly.

1. Go to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Change the value of am.ehcache.replicator.factory to "JMS":

Properties (.properties files)

```
# EhCache Cache Replicator Factory: RMI(default) or JMS
# If JMS is used, related properties(starting with
am.ehcache.jms) must be enabled.
am.ehcache.replicator.factory=JMS
```

a. If using MQ Service, go to the *<ACMATRIX_ROOT>\sdk\ActiveMQ\lib* directory and copy **all the jar files** to *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\lib* folder, then uncomment the following in **amsystem.properties** file and set the values accordingly:

Properties (.properties files)

```
# EhCache JMS Cache Peer Provider (MQ-like service: Apache
ActiveMQ, AWS MQ, etc...)
am.ehcache.jms.initialContextFactoryName=com.isprint.am.core.c
ache.ehcache.AtiveMQInitialContextFactory
am.ehcache.jms.providerURL=failover:tcp://localhost:61616
am.ehcache.jms.userName=admin
am.ehcache.jms.password=admin
am.ehcache.jms.replicationTopicBindingName=amtopic
am.ehcache.jms.getQueueBindingName=amqueue
#am.ehcache.jms.acknowledgementMode
#       : (Optional) The mode used to acknowledge messages
after they are consumed.
#    Configure with one of the following values:
#         - AUTO_ACKNOWLEDGE
#         - DUPS_OK_ACKNOWLEDGE (Default)
#         - SESSION_TRANSACTED
#am.ehcache.jms.acknowledgementMode=AUTO_ACKNOWLEDGE
```

Additionally, configure the ActiveMQ trusted package by going to the directory *<ACMATRIX_ROOT>\tomcat\bin* and opening the file **setenv.bat** or **setenv.sh** (depending on your OS):

<table>
<tr>
<td>Windows</td>
<td>Open setenv.bat, search for :setJavaOpts and add the line set JAVA_OPTS=%JAVA_OPTS
Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*.
For example:
<table>
<tr><th>Code</th></tr>
<tr><td>

```
:setJavaOpts
set TITLE=AccessMatrix Server
set
"LOCALLIBPATH=%JAVA_HOME_LIBPATH%;bin;%PATH%;%LOCALLIBPATH%"
set JAVA_OPTS=%JAVA_OPTS% -
Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*
```

</td></tr>
</table>
</td>
</tr>
<tr>
<td>Linux</td>
<td>Open setenv.sh, search for JAVA_OPTS and add the line -
Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*.
For example:
<table>
<tr><th>Bash (Unix Shell)</th></tr>
<tr><td>

```
JAVA_OPTS="$JAVA_OPTS $JAVA_GC_OPTS -Xms400m -Xmx1g -
Dderby.system.home=db
-Dam.ehcache.set_rmi_ip_addr=true -
XX:MaxMetaspaceSize=256M -XX:NewSize=128M
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=logs -
```

</td></tr>
</table>
</td>
</tr>
</table>

```
Dhost.name=$(cat /etc/hostname)
-Djavax.net.ssl.trustStore=conf/amtrust.keystore -
Djavax.net.ssl.trustStorePassword=passphrase
-Djava.awt.headless=true" -
Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*
```

b. If using Azure Service Bus, go to the *<ACMATRIX_ROOT>\sdk\AzureServiceBus\qpid-libs* directory and copy all the jar files to the *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\lib* folder, then uncomment the following in **amsystem.properties** file and set the values accordingly:

Properties (.properties files)

```
# == EhCache JMS Peer Provider (ONLY If using Azure Service
Bus) ==
# Other am.ehcache.jms properties must be disabled
am.ehcache.jms.azuresb.providerURL=failover:(amqps://<service_
bus_namespace_name>.servicebus.windows.net?amqp.idleTimeout=12
0000&amp;amqp.traceFrames=true)
am.ehcache.jms.azuresb.securityPrincipalName=<servicebus_share
d_access_policy_name>
am.ehcache.jms.azuresb.securityCredentials=<servicebus_shared_
access_policy_key>
am.ehcache.jms.azuresb.tenantId=<tenant_id>
am.ehcache.jms.azuresb.subscriptionId=<subscription_id>
am.ehcache.jms.azuresb.client_id=<your_client_id>
am.ehcache.jms.azuresb.client_secret=<your_client_secret>
am.ehcache.jms.azuresb.namespaceName=<service_bus_namespace_na
me>
am.ehcache.jms.azuresb.resourceGroupName=<resource_group_name>
am.ehcache.jms.azuresb.topicName=<topic_name>
# Time interval (seconds) of generating cache to send to Azure
Service Bus for heartbeat purpose. Default is 300.
am.ehcache.jms.azuresb.heartbeat=300
```

Ehcache Synchronous/Asynchronous Replication

By default, Ehcache replication is done asynchronously. That means if the cache is modified in an instance, the modification is not blocked. The replication will be done by a background thread that periodically runs to replicate the pending events. If it is configured to be synchronous, then the modification of a cache is blocked until replication is done.

A general rule is not to use synchronous since it will, for sure affect performance. The asynchronous replication event is by default done within 100 milliseconds (background thread runs within 100 milliseconds to start replication task), and it can be configured for each cache. However, although it is unlikely, for unique caches like SMS-OTP Memory Token Store and E2EEA sessions, it may get some

issues when the cache entries are not replicated yet. To mitigate this, synchronous replication can be used when that becomes a problem.

SMS-OTP needs to send the OTP via SMS or Email, and this will take a few seconds for the user to receive and another a few seconds for the user to type. It is similar to E2EE where it will take some time for the user to enter a password. Thus, it is still recommended to set it to asynchronous unless the cache is not replicated in time when the problem happens too often. Even for this, it should be verified that the AccessMatrix Ehcache peer discovery is set to multicast. When Ehcache peer discovery is set to "DB" (the default value), an offline server may be the one that is causing the delay in replication.

The asynchronous replication interval can also be configured for each cache instead of using its default value of one second by configuring the following cacheEventListenerFactory property:

Properties (.properties files)

```
    * replicateAsynchronously=true | false - whether
replications are
      asynchronous (true) or synchronous (false).
Defaults to true.

    * asynchronousReplicationIntervalMillis=<number of
milliseconds> - The asynchronous replicator runs at a
set interval of milliseconds. The default value is 100
(ms)
      and the minimum is 10 (ms). However, if no value
is set, then the asynchronous replicator runs at a set
interval of 1000 (ms). This property is only applicable
if
      replicateAsynchronously=true
```

Redis PubSub

Redis PubSub (Publish/Subscribe) is a messaging pattern supported by Redis. It specifies a method of message communication between multiple parties, each defined as either a publisher or subscriber:

- **Publishers** - They publish (send messages) to specific channels. Channels are the communication pathway between publishers and subscribers.
- **Subscribers** - They subscribe to (opt in to receive messages from) channels. Subscribers only receive messages from channels they are subscribed to.

Publishers do not send messages to specific subscribers, but rather to channels. This means that publishers do not require knowledge about any subscribers. Likewise, subscribers can select which

channels they want to receive messages from and disregard any other channels. As a result, the management of the information being communicated is greatly simplified.

In the context of cache replication, an AccessMatrix instance acts as the publisher, while other instances being replicated to act as subscribers. The Redis server provides the channel through which messages are communicated.

To implement Redis PubSub for cache replication, the AccessMatrix server must first be connected to a Redis server. Refer to AccessMatrix Server to Redis Server Connection for details on establishing a secure connection between them.

Next, the administrator or application developer has to configure **amsystem.properties**:

1. Go to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Search for the Ehcache section:

   Properties (.properties files)

   ```
   # == Ehcache ==
   # EhCache Cache Replicator Factory: RMI(default) or JMS
   or Redis
   # If JMS is used, related properties(starting with
   am.ehcache.jms) must be enabled.
   am.ehcache.replicator.factory=RMI
   # If Redis is used
   am.ehcache.replicator.redis.channel = ampubsub
   am.ehcache.replicator.redis.reconnect.secs = 10
   ```

4. Update the value of #am.ehcache.replicator.factory from "RMI" to "Redis":

   Properties (.properties files)

   ```
   # == Ehcache ==
   # EhCache Cache Replicator Factory: RMI(default) or JMS
   or Redis
   # If JMS is used, related properties(starting with
   am.ehcache.jms) must be enabled.
   am.ehcache.replicator.factory=Redis
   # If Redis is used
   am.ehcache.replicator.redis.channel = ampubsub
   am.ehcache.replicator.redis.reconnect.secs = 10
   ```

5. Modify the am.ehcache.replicator.redis.reconnect.secs setting, if needed. This setting determines how long the AccessMatrix server should wait before attempting to reconnect to the Redis server.

Configure Special Caches

Caches used by Hibernate, CachedObjectRepository, and most other modules are already preconfigured for replication. However, some caches are not configured for replication by default. Depending on the deployment architecture, you may need to configure the following caches described in this section for replication.

To configure the special caches, perform the following:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **am5-ehcache.xml** file.
3. Search for the relevant cache configuration (e.g. "MemoryTokenstore"):

XML

```
<cache name="MemoryTokenStore"
    maxElementsInMemory="30000"
    eternal="false"
    timeToIdleSeconds="0"
    timeToLiveSeconds="900"
    overflowToDisk="false">
    <!-- if generateOtp/verifyOtp could end up with different
instances of AM5 servers, replication must be enabled and tested
-->

    <cacheEventListenerFactory
class="com.isprint.am.core.cache.ehcache.CacheReplicatorFactory"
       properties="replicateAsynchronously=true,
                   replicatePuts=true,
                   replicateUpdates=true,
                   replicateUpdatesViaCopy=true,
                   replicateRemovals=true,
                   asynchronousReplicationIntervalMillis=100"/>
</cache>
```

4. Update the parameters as required and ensure that it has been uncommented.
5. Save the changes.

UAS - SMS-OTP Memory Cache

By default, SMS OTP uses MemoryTokenStore which does not persist in the database. If the project requires session stickiness that can ensure that the same AccessMatrix server serves generateOtp and verifyOtp, replication is not required. This configuration ensures greater efficiency.

However, if there is no session stickiness and no cache replication, verifyOtp may be served by a separate AccessMatrix server instance that is unable to verify the missing cache entry. This causes an issue such as tokens not to be found. If SystemTokenStore is used instead of MemoryTokenStore, there is no need to configure cache replication for MemoryTokenStore as this cache will not be used.

By default, cache replication has been configured to allow generateOTP and verifyOTP to be performed by separate AccessMatrix server instances.

UAS - E2EE Session Cache

E2EE Authentication requires the E2EE session to be generated before every login/change password/reset password. Each E2EE session can only be used once to avoid a replay attack. AccessMatrix stores the E2EE session in the cache. As part of E2EE session validation, AccessMatrix retrieves the session information from the cache to ensure that the session exists/valid. E2EE session cache expiry is configured via Admin Console's Global Setting.

If the deployment has session stickiness that ensures the E2EE preauthentication (E2EE session generation) and E2EE authentication/change password/reset the same AccessMatrix server does password (E2EE session consumption), there is no need to configure replication.

By default, cache replication has been configured to allow the E2EE session to be generated and validated across separate AccessMatrix server instances.

RADIUS Session Cache

RADIUS server cache is required if RADIUS challenge-response protocol is used. It is defined as com.isprint.am.core.auth.EhCacheSessionRepository.sessionCache.

No replication is necessary as, usually, the RADIUS client will stick to the same RADIUS server during the entire RADIUS challenge-response session.

FIDO2 Request Cache

In FIDO2 authentication, the initiation of each registration or assertion ceremony requires a unique FIDO2 challenge to be generated by the AccessMatrix server. The FIDO2 challenge is subsequently cached and used to validate and complete the registration or assertion process. This ensures that the

FIDO2 authenticator has a valid interaction with the Relying Party, and replay attacks attempting to imitate the FIDO2 challenge will not be successful.

> **Note:** The FIDO2 challenge can be configured to persist in the database instead. For more details, refer to the UAS FIDO2 Token Implementation Guide - Housekeeping FIDO2 Challenge Stored in Database.

By default, cache replication has been configured to allow the FIDO2 challenge to be generated and used for validation across separate AccessMatrix server instances.

Ehcache Troubleshooting

Logging

Ehcache comes with logging capabilities that can help in the troubleshooting process, especially for cache replication issues. Ehcache uses a commons-logging module. For Apache Tomcat, the log can be configured in Tomcat's **logging.properties**. For bundled Tomcat, the **logging.properties** contains the entries for Ehcache. To enable it, you need to change the log level to "FINE"/"FINEST"/"ALL".

Bundled Tomcat's logging.properties

Properties (.properties files)

```
# Change below to FINE for debugging of ehcache replication
net.sf.ehcache.level = INFO
net.sf.ehcache.handlers
=5ehcache.org.apache.juli.AsyncFileHandler
```

> **Note**: Ehcache logging will generate a lot of entries. Thus, make sure that there is enough free disk space. This debug logging should only be enabled for troubleshooting purpose. For regular operation, it can slow down the system by a large amount of generated log entries.

SMS-OTP MemoryTokenStore/E2EE Session/RADIUS Session Caches Logging

To enable debugging log for SMS-OTP MemoryTokenStore/E2EE Session/RADIUS Session caches, modify **amlog4j2.properties** to set the log level to "DEBUG" for the following:

Properties (.properties files)

```
# Change the following caches into DEBUG to print output/remove/eviction
per element for troubleshooting
# E2EE temp session cache
```

```
logger.COM_ISPRINT_AM_CORE_AUTH_E2EESESSIONREPOSITORY.name =
com.isprint.am.core.auth.E2EESessionRepository
logger.COM_ISPRINT_AM_CORE_AUTH_E2EESESSIONREPOSITORY.level = DEBUG
logger.COM_ISPRINT_AM_CORE_AUTH_E2EESESSIONREPOSITORY.additivity =false
logger.COM_ISPRINT_AM_CORE_AUTH_E2EESESSIONREPOSITORY.appenderRef.rolling
.ref = LOG_ROOT
# USO / RADIUS short session id cache
logger.COM_ISPRINT_AM_CORE_AUTH_EHCACHESESSIONREPOSITORY.name =
com.isprint.am.core.auth.EhCacheSessionRepository
logger.COM_ISPRINT_AM_CORE_AUTH_EHCACHESESSIONREPOSITORY.level = DEBUG
logger.COM_ISPRINT_AM_CORE_AUTH_EHCACHESESSIONREPOSITORY.additivity
=false
logger.COM_ISPRINT_AM_CORE_AUTH_EHCACHESESSIONREPOSITORY.appenderRef.roll
ing.ref = LOG_ROOT
# SMS OTP Cache
logger.COM_ISPRINT_AM_CORE_TOKENMGMT_MEMORYTOKENSTORE.name =
com.isprint.am.core.tokenmgmt.MemoryTokenStore
logger.COM_ISPRINT_AM_CORE_TOKENMGMT_MEMORYTOKENSTORE.level = DEBUG
logger.COM_ISPRINT_AM_CORE_TOKENMGMT_MEMORYTOKENSTORE.additivity =false
logger.COM_ISPRINT_AM_CORE_TOKENMGMT_MEMORYTOKENSTORE.appenderRef.rolling
.ref = LOG_ROOT
```

Ehcache Monitoring

You can monitor how caches are utilized (e.g. number of cache hits/misses, size) by using JMX monitoring tool. First, you need to enable the JMX feature of the web app server. Once you have done so, you can enable JMX monitoring for Ehcache by configuring **amsystem.properties** as follows:

Properties (.properties files)

```
am.ehcache.jmx=true
```

Using the JMX monitoring tool, you will be able to see the cache statistics MBean for each cache.

Testing Replication

Modify a user using AdminConsole:

1. Log in to the Admin Consoles of AccessMatrix servers 1 and 2. On both Admin Consoles, go to the user search form, search all users/a particular user and view a user (referred to as User A). This is to make Hibernate load User A into the cache.
2. Using the Admin Console of AccessMatrix server 1, edit User A to change an attribute/description, then save.

3. Using the Admin Console of AccessMatrix server 2, click on the **Back** button so that it shows the previous search result (do not try to search again), and then view User A again. This is to refresh the page to see whether the replication has taken effect.
4. If the change done by AccessMatrix server 1 is reflected in AccessMatrix server 2, that means the replication is working.
5. Repeat the process, but change the direction (use AccessMatrix server 2 to edit user A) to make sure that the replication works in both directions.

Another way to know whether replication is working is to analyze the Ehcache log.

Log Analysis

Log analysis of Ehcache log:

- **For Peer Registration**
  - Search this keyword "RMICacheManager"
- **For Peer Discovery Setting**
  - Search this keyword "peerDiscovery"
- **For Replication Event (from another instance)**
  - Search this keyword "RMICachePeer"

Issues

Replication does not take place or is slow

If AccessMatrix Ehcache Peer Discovery is set to DB (the default value), the list of servers configured in Admin Console will be used as the list of replication peers. It would help if you made sure that all servers listed are running. If any of the servers are not running, then it may delay the replication process. In manual Ehcache replication, Ehcache will try to replicate to the configured peers in a blocking manner. That means it will try to replicate to peer1, then peer2, and so on. If peer1 is not running, the Ehcache replication process may need to wait until it hits a TCP timeout before it continues to the next peer. The order of the peers is not guaranteed.

The best solution is to use multicast. In multicast, the list of peers is automatically updated based on the heartbeat sent to the multicast group.

If there are servers listed in Admin Console that are not used anymore, then they can also be deleted.

Other known replication issues

Check if the error java.rmi.NoSuchObjectException: no such object in table is encountered in the **am.log** file:

```
et.sf.ehcache.distribution.RMIAsynchronousCacheReplicator.flushReplicationQueue Unable to send
message to remote peer. Message was: no such object in table
   java.rmi.NoSuchObjectException: no such object in table
      at
java.rmi/sun.rmi.transport.StreamRemoteCall.exceptionReceivedFromServer(StreamRemoteCall.java:303)
      at java.rmi/sun.rmi.transport.StreamRemoteCall.executeCall(StreamRemoteCall.java:279)
      at java.rmi/sun.rmi.server.UnicastRef.invoke(UnicastRef.java:164)
      at net.sf.ehcache.distribution.RMICachePeer_Stub.send(Unknown Source)
```

If you encounter this error, you can attempt to resolve the issue by performing the following:

1. Navigate to *<AM_ROOT>\tomcat\webapps\am5\WEB-INF\classes* and open the **amsystem.properties** file.
2. Search for the line am.ehcache.ip_addr:

   Properties (.properties files)

   ```
   # am.ehcache.ip_addr
   #   : Ehcache RMI listener IP address. Applicable when
   the host has multiple IPs and primary IP is not set to
   the correct IP.
   #     AM normally will automatically configure this
   (e.g. if host as has multiple IPs, but the primary IP
   is
   #     not the same as IP address configured in AM
   Server object)
   #     If there is a need to override the listener IP
   address, then this property can be set to the correct
   IP.
   #     (This setting is applicable only if
   peer_discovery=multicast/db)
   ```

3. Uncomment the field and set the value to the AccessMatrix IP address. For example:

   Properties (.properties files)

   ```
    am.ehcache.ip_addr= 10.171.1.255
   #   : Ehcache RMI listener IP address. Applicable when
   the host has multiple IPs and primary IP is not set to
   the correct IP.
   #     AM normally will automatically configure this
   (e.g. if host as has multiple IPs, but the primary IP
   ```

```
is
#     not the same as IP address configured in AM
Server object)
#     If there is a need to override the listener IP
address, then this property can be set to the correct
IP.
#     (This setting is applicable only if
peer_discovery=multicast/db)
```

> **Note:** This must be the IP address of your AccessMatrix server, *not* the IP address of the remote AccessMatrix server.

4. Save the changes and restart the server.

SMS-OTP request, RADIUS/E2EE sessions are randomly removed before they are used

Cache has a maximum capacity (number of entries to cache). The capacity can be configured from the Ehcache configuration file (**am5-ehcache.xml**). If the cache is full, Ehcache will remove the entry to accommodate a new one. Ehcache LRU (Least Recently Used) algorithm is non-deterministic, which may result in random cache entry get evicted. To make the Ehcache eviction behaves in deterministic LRU, this configuration can be added to Java System Properties or **amsystem.properties**:

Properties (.properties files)

```
net.sf.ehcache.use.classic.lru=true
```

Usually, the default cache capacity for SMS-OTP requests and RADIUS/E2EE sessions are large enough. However, if the project expects more active SMS-OTP requests or RADIUS/E2EE sessions, then the cache size can be adjusted.

- E2EEA sessions cache name: com.isprint.am.core.auth.E2EESessionRepository.sessionCache
- SMS-OTP cache name: MemoryTokenStore
- RADIUS challenge response sessions cache name: com.isprint.am.core.auth.EhCacheSessionRepository.sessionCache

Caches in Kubernetes or Cloud Environment

In Kubernetes and in cloud environments, AccessMatrix caches need to be suitably configured for each environment:

- Use JMS Replication instead of RMI Replication to configure Ehcache replication. Alternatively, use Redis PubSub for cache replication.
- Use Redis Cache Servers for AccessMatrix caches under the category of "Transient Data Primary Storage Cache". Please refer to AccessMatrix Server Connects to Redis Server.
- Keep hibernate cache disabled (disabled by default).

With the above configurations, AccessMatrix will use two cache technologies: Ehcache and Redis. Ehcache involves in-memory caching and is faster than Redis. Hence, Ehcache is used to cache important objects, such as policy, configuration, login modules, etc. The replication of these objects is done using JMS. In this configuration, Redis replaces Ehcache for caches acting as the primary storage for transient data.

3.10. Advanced Configurations

AccessMatrix provides Advanced Configurations to cater to enterprises' individual needs.

Configure SM Cryptography

Configuration

SM (formerly known as SMS -Synchronous Meteorological Satellite) Cryptography is a Chinese national cryptographic algorithm standard that consists of SM2, SM3, and SM4. SM2 is a signature cryptographic algorithm, SM3 is a MessageDigest hash cryptographic algorithm and SM4 is a Symmetric Block Cipher cryptographic algorithm. AM5 supports SM3 and SM4 only. SM3 is used for SystemPasswordBasic Hash Scheme and Oath Token Suite whereas SM4 is used for Key Encryption.

> **Important Note:** SM Cryptography is only for deployment in China.

Deployment

1. Obtain the library (e.g. **mysmprovider.jar**) and place it to this location: *<AxMx Root>/webapps/am5/WEB-INF/lib/*.
2. Restart AccessMatrix server.

> **Note:** There is no warning in **am.log** if this library is not properly loaded.

Confirm SM3 Support

1. Log in to the Admin Console.
2. Select the **Administration Dashboard** from the Dashboard drop-down list at the upper right corner of the first-level navigation bar.
3. Navigate to **Configuration > Authentication > Login Module** and select the "SystemPasswordBasic" login module.
4. Click the **Edit** button. Select "Salted SM3" under the **Hash scheme** attribute.

   > **Note:** If Salted SM3 is not available, it means the library is not properly loaded.

Confirm SM4 Support

1. Log in to the Admin Console.

2. Navigate to **Administration Dashboard > Security Policy > Key Management > Key Manager > Key Provider**. **Search Key Provider** will be displayed on the right pane.
3. Click **Create** to display the **Choose Key Provider Implementation** page.
4. Select "SM Key Provider" from the **Implementation** list and click **Next** to display the **Create Key Provider** page.
5. Enter the desired **Key Provider Id**, **Key Provider Name**, and click **Save**.
6. On the left pane, select **Encryption Key** to display the **Search Encryption Key** page.
7. Click **Create**. The **Choose Encryption Key Template** page is displayed.
8. Select the key provider created before from the **Key Provider** list and select "Symmetric key (secret key)" from **Encryption key template** list. Then click **Next** to display the **Create Encryption Key** page.
9. Enter the desired **Encryption Key Id**, **Encryption Key Name** and other required information. Click **Save**.

> **Note:** The creation of the encryption key will fail if the library is not properly loaded.

Configure Quartz Scheduler

AccessMatrix is using Quartz Scheduler to handle background tasks. To configure Quartz Scheduler, ensure that the following configuration is added in **amsystem.properties**.

> **Note:** The configuration below already includes support for the UCM Video Converter, as mentioned in the *UCM Deployment Guide*.

Properties (.properties files)

```
# # == Quartz Scheduler ==
am.scheduler.names=local,default-cluster,ucm-cluster
am.scheduler.local.scheduler.instanceName=local
am.scheduler.local.scheduler.instanceIdGenerator.class=com.isprint.am.cor
e.scheduler.AMInstanceIdGenerator
am.scheduler.local.scheduler.instanceId=AUTO
am.scheduler.local.threadPool.threadCount=5
am.scheduler.local.jobStore.class=org.quartz.simpl.RAMJobStore
am.scheduler.default-cluster.scheduler.instanceName=default-cluster
am.scheduler.default-
cluster.scheduler.instanceIdGenerator.class=com.isprint.am.core.scheduler
.AMInstanceIdGenerator
am.scheduler.default-cluster.scheduler.instanceId=AUTO
am.scheduler.default-cluster.threadPool.threadCount=5
am.scheduler.default-cluster.jobStore.isClustered=true
```

```
am.scheduler.default-
cluster.jobStore.class=org.quartz.impl.jdbcjobstore.JobStoreTX
am.scheduler.ucm-cluster.scheduler.instanceName=ucm-cluster
am.scheduler.ucm-
cluster.scheduler.instanceIdGenerator.class=com.isprint.am.core.scheduler
.AMInstanceIdGenerator
am.scheduler.ucm-cluster.scheduler.instanceId=AUTO
am.scheduler.ucm-cluster.threadPool.threadCount=5
am.scheduler.ucm-cluster.jobStore.isClustered=true
am.scheduler.ucm-
cluster.jobStore.class=org.quartz.impl.jdbcjobstore.JobStoreTX
#am.scheduler.default-cluster.scheduler.standby=true
```

Note: Setting am.scheduler.default-cluster.scheduler.standby to "true" will enable the standby mode property, which allows the AccessMatrix Server instance to add a job to the scheduler (but not take part in executing any job). The default setting is "false".

Configure Support for Non-English Characters

1. Go to the directory *<ACMATRIX_ROOT>\tomcat\bin* and open the **setenv.bat** file.
2. Add -Dfile.encoding=utf-8 to the following setting to support non-English characters, e.g. Chinese characters:

Code

```
set JAVA_OPTS=%JAVA_OPTS% "-Dfile.encoding=utf-8" -
Xms400m -Xmx1g "-Djava.library.path=%LOCALLIBPATH%" -
Dderby.system.home=db -XX:MaxMetaspaceSize=256M -
XX:NewSize=128M -Xloggc:logs/gc.log -XX:+PrintGCDetails
-XX:+DisableExplicitGC -XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=logs -Xincgc
```

Configure Session Token Cookie Name

If the USO Portal, CLP, or WSA RP is used and deployed under the same domain as the AccessMatrix Admin Console, you are strongly advised to change the default cookie name (WSASID) of the Admin Console. This is to prevent unintended issues arising from cookie conflicts (e.g. cookie overwriting, path conflict, domain conflict, Secure/HttpOnly conflict).

The AccessMatrix server's session token cookie name can be customized as needed. To do so:

1. Navigate to *<AM_ROOT>\tomcat\webapps\am5\WEB-INF\classes* and open the **wpe_config.properties** file.

2. Search for the authentication.wsasid.cookie.name setting:

Properties (.properties files)

```
#AM-11748 session token cookie name, default value is
WSASID
authentication.wsasid.cookie.name = WSASID
```

3. Replace the default value with an appropriate name (e.g. "AM5ServerCookie").
4. Save the changes.

4. AccessMatrix Maintenance and Upgrade

The following pages provide detailed information on the maintenance of the AccessMatrix system:

> **Note:** For more information on how to upgrade the AccessMatrix system, refer to the AccessMatrix Product Upgrade Guide.

- Maintenance of AccessMatrix System
- Monitoring the AccessMatrix System

4.1. Maintenance of the AccessMatrix System

There may be situations where the administrator is requested to configure certain component's settings to cater to specific user requirement. See below for some basic maintenance tasks the administrator should perform on the AccessMatrix system to ensure quick system recovery in the case of any system crash.

- **AccessMatrix Full System Backup/Restore**
    - This refers to backing-up or restoring the whole AccessMatrix system. AccessMatrix full system backup is highly recommended to be performed at least once after completing the installation and configuration of the entire system.
    - Subsequently, full system backup should be performed whenever there are changes made to the system configuration. This is important in case of disaster recovery so as to minimize system downtime.

      Note: For AccessMatrix distributed system, the AccessMatrix Web Archive backup should be performed.
- **AccessMatrix Security Registry Backup/Restore**

  This refers to backing-up or restoring the AccessMatrix security registry only. Since this is the only AccessMatrix component that contains dynamic data and changes in every day's operation, full security registry backup should be performed at the end of the day to ensure the backup copy is always up-to-date. This is critical in the case of database recovery which could be due to database corruption or failure.

Backup AccessMatrix and Database

AccessMatrix Dispersed System	1. Stop AccessMatrix Server if it is running. 2. Open the Windows Explorer (for Windows) or terminal (for Unix) and navigate to the AccessMatrix root or installation directory, i.e. <*ACMATRIX_ROOT*>. 3. Backup the whole installation folder (including all sub-folders and files beneath it). 4. Keep this backup copy in a folder outside of the installation directory, in another server, or in a backup media. 5. After finish backing-up the system, start the AccessMatrix Server.
AccessMatrix Distributed System	To back up the AccessMatrix Web Archive:

	1. Open the Windows Explorer and navigate to the AccessMatrix installation directory, i.e. <*ACMATRIX_ROOT*>. 2. Backup the AccessMatrix Web Archive, **am5.war**. 3. Keep this backup copy in a folder outside of the installation directory, in another server, or in a backup media.

Back up Security Registry

AccessMatrix Dispersed/Distributed System

Since the AccessMatrix security registry resides in a dispersed database server, the system administrator should consult the database administrator on how to perform a full or incremental database backup of the AccessMatrix security registry. Prior to performing security registry backup, the Administrator should stop the AccessMatrix server first if it is running. Restart AccessMatrix Server once done.

Restore AccessMatrix and Database

AccessMatrix Dispersed System	1. Stop AccessMatrix Server if it is running. 2. Open the Windows Explorer (for Windows) or terminal (for Unix) and navigate to the AccessMatrix root or installation directory, i.e. <*ACMATRIX_ROOT*>. 3. Restore the whole installation folder (including all sub-folders and files beneath it) from the latest backup copy. **Note**: If the administrator is clear about the requested files to be restored, a partial restore may be performed, i.e. restore only those affected files. 4. Restore the latest backup copy of AccessMatrix security registry since the full restore will overwrite the existing security registry. 5. After you have completed restoring the system, start the AccessMatrix Server.
AccessMatrix Distributed System	To restore the AccessMatrix Web Archive: 1. Open the Windows Explorer and navigate to the AccessMatrix installation directory, i.e. <*ACMATRIX_ROOT*>. 2. Restore the AccessMatrix Web Archive from the latest backup copy.

Restore Security Registry

AccessMatrix Dispersed/Distributed System

Since the AccessMatrix security registry resides in a dispersed database server, the system administrator should consult the database administrator on how to perform a full or incremental database restore of the AccessMatrix security registry. Prior to performing security registry backup, the Administrator should stop the AccessMatrix server first if it is running. Restart AccessMatrix Server once done.

4.2. Monitoring the AccessMatrix System

AccessMatrix provides monitoring reports within the Admin Console that provide useful system and server information. Such information may be important for stress tests in non-production environments, or to monitor the system within production environments. To access the monitoring report page, navigate to **Report Dashboard > System & Admin > Real Time > Monitoring** in the Admin Console.

Refer to AccessMatrix Common Administration Guide - Server Monitoring for more details on configuring the monitoring report items.

Prometheus Monitoring

Other than the default monitoring report feature, AccessMatrix also supports the use of Prometheus, an open-source systems monitoring and alerting toolkit that is widely used. To integrate AccessMatrix with Prometheus, use the endpoint /am5/metrics, which exposes metrics for Prometheus to scrap/collect. To generate more reader-friendly monitoring reports, an open source analytics and monitoring solution like Grafana can be used together with Prometheus to transform the metrics into insightful graphs and visualizations.

Example of metrics available from /am5/metrics endpoint

Code

```
# HELP tomcat_thread_count Tomcat Thread Count
# TYPE tomcat_thread_count gauge
tomcat_thread_count{state="current",} 0.0
tomcat_thread_count{state="busy",} 0.0
# HELP am_average_authentication_response_time_ms AM
Average Authentication Response Time (ms)
# TYPE am_average_authentication_response_time_ms gauge
am_average_authentication_response_time_ms 0.0
# HELP jvm_maximum_memory_mb JVM Maximum Memory (mb)
# TYPE jvm_maximum_memory_mb gauge
jvm_maximum_memory_mb 8116.0
# HELP tomcat_dbcp_size Tomcat Database Connection Pool
Size
# TYPE tomcat_dbcp_size gauge
tomcat_dbcp_size 0.0
# HELP am_nth_percentile_response_time_ms AM 90th
Percentile Authentication Response Time (ms)
# TYPE am_nth_percentile_response_time_ms gauge
am_nth_percentile_response_time_ms 0.0
```

```
# HELP tomcat_dbcp_current_connections_count Tomcat
Database Connection Pool Current Connection Count
# TYPE tomcat_dbcp_current_connections_count gauge
tomcat_dbcp_current_connections_count{state="idle",}
0.0
tomcat_dbcp_current_connections_count{state="active",}
0.0
# HELP jvm_thread_count JVM Thread Count
# TYPE jvm_thread_count gauge
jvm_thread_count 187.0
# HELP jvm_free_memory_mb JVM Free Memory (mb)
# TYPE jvm_free_memory_mb gauge
jvm_free_memory_mb 1400.0
# HELP am_event_thread_pool_active AM Event Thread Pool
Active Count
# TYPE am_event_thread_pool_active gauge
am_event_thread_pool_active 0.0
# HELP am_event_thread_pool_rejected AM Event Thread
Pool Rejected Count
# TYPE am_event_thread_pool_rejected gauge
am_event_thread_pool_rejected 0.0
# HELP am_event_thread_pool_queue_size AM Event Thread
Pool Queue Size
# TYPE am_event_thread_pool_queue_size gauge
am_event_thread_pool_queue_size 0.0
# HELP am_event_thread_pool_rejected_window AM Event
Thread Pool Rejected Count in Time Window
# TYPE am_event_thread_pool_rejected_window gauge
am_event_thread_pool_rejected_window 0.0
```

5. Encryptor Utility Tool

The Encryptor Utility Tool provides options to encrypt the truststore, keystore and database passwords.

To use the Encryptor Utility:

1. Open command prompt and run the **encryptor.bat** file from *<AccessMatrix_Installation_Directory>\bin* folder. A menu will be displayed.

   > **Note:** Run the **encryptor.sh** file when using Linux machine.

2. Select the desired encryption scheme from the menu.

```
- Main Menu -

0. Exit
1. Aes256Encryptor
2. AesEncryptor
3. HashicorpVaultEncryptor
4. HostBasedAesEncryptor

Enter selection (default: [0]) -->
```

3. Type the equivalent menu value and press the Enter key.

It is recommended to use only the Aes256Encryptor, AesEncryptor, and HostBasedAesEncryptor algorithms, as others are insecure. For Aes256Encryptor, ensure that the JRE used by this utility and the AccessMatrix Server is using an unrestricted JCE policy to allow AES 256-bit key.

When choosing Aes256Encryptor or AesEncryptor, it is possible to use a file or an environment variable to store the "user salt" that will be used to derive the encryption key. If a file is used, ensure that the file permission is set to allow only OS user used to run AccessMatrix Server and Encryptor to have access of the file containing the user salt. Using a file to store the "user salt" is preferred as this is to avoid of accidental disclosure when sending configuration files for troubleshooting or other purposes.

Using AesEncryptor to Encrypt Database Password

Perform the following steps to encrypt the database password for Tomcat/DBCP:

1. Prepare a text file containing a random string that will be used as user salt and place it in a secure directory and restrict the file access to allow only OS user used to run AccessMatrix Server and the Encryptor Utility Tool.
2. Run **encryptor.bat** and type "2" to select `AesEncryptor`, then press the Enter key. The AesEncryptor menu is displayed.

```
- Menu: 2 - Encryptor Menu -

- EncryptorId: AesEncryptor


0. Back to main
1. Encrypt a passphrase
2. Enter a user salt for this encryptor
3. Generate a new random user salt
4. Use a file as user salt for this encryptor
5. Use an environment variable as user salt for this encryptor
```

3. Type "4" to select `Use a file as user salt for this encryptor` and press the Enter key.
4. The screen will ask for the file path. Type the location of the file and press the Enter key.

```
Enter file path --> /home/am/conf-secure/encryptor-secret.txt

New EncryptorId: AesEncryptor::file=/home/am/conf-secure/encryptor-secret.txt

Press [Enter] to continue ...
```

5. Back at the screen of `AesEncryptor`, type "1" to select `Encrypt a passphrase` and press the Enter key.
6. Type the passphrase or password to be encrypted and re-type the passphrase or password to be encrypted for confirmation. It will display the resulting ciphertext in SingleFieldEncryptedValue.

```
- Menu: 2 - Encryptor Menu -

- EncryptorId: AesEncryptor::file=/home/am/conf-secure/encryptor-secret.txt


0. Back to main
1. Encrypt a passphrase
2. Enter a user salt for this encryptor
3. Generate a new random user salt
4. Use a file as user salt for this encryptor
5. Use an environment variable as user salt for this encryptor

Enter selection (default: [0]) --> 1


Enter passphrase to be encrypted (default: [passphrase]) -->**********


Confirm passphrase to be encrypted -->**********

Encrypted passphrase: L3mF8dQxiA/iVUrNnv+GR9VUljI5dGnyMKFg9nad2NA=
EncryptorId: AesEncryptor::file=/home/am/conf-secure/encryptor-secret.txt

SingleFieldEncryptedValue:
AesEncryptor::file=/home/am/conf-secure/encryptor-
secret.txt:L3mF8dQxiA/iVUrNnv+GR9VUljI5dGnyMKFg9nad2NA=

Press [Enter] to continue ...
```

7. Copy the encrypted passphrase or password under `SingleFieldEncryptedValue`.
8. Open the **am5.xml** file from the *<ACMATRIX_ROOT>\tomcat\conf\Catalina\localhost* folder.
9. Modify the **am5.xml** file by supplying the encrypted value to the password attribute as shown in the example below:

```
XML

<Resourcename="jdbc/DSamdb"auth="Container"type="javax.sql.DataSour
ce"description="AM5 DB"
factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory"
initialSize="10"maxTotal="80"maxIdle="20"minIdle="10"maxWaitMillis=
"10000"
validationQuery="SELECT COUNT( * ) FROM am_vars WHERE va_name IS
NULL"testOnBorrow="true"
username="user1"password="AesEncryptor::file=/home/am/conf-
secure/encryptor-
secret.txt:L3mF8dQxiA/iVUrNnv+GR9VU1jI5dGnyMKFg9nad2NA="driverClass
Name="org.apache.derby.jdbc.EmbeddedDriver"
url="jdbc:derby:amdb;create=true"
/>
```

10. Do take note that an additional parameter is added: factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory".

Using AesEncryptor to Encrypt TrustStore and KeyStore Password

If the user is using SSL connection for the DB connector, then the passphrase for the keystore and truststore should be encrypted as well.

1. Encrypt the keystore and truststore passphrase by following steps 1 to 7 from Using AesEncryptor to Encrypt Database Password section.
2. Open the **am5.xml** file from the *<ACMATRIX_ROOT>\tomcat\conf\Catalina\localhost* folder.
3. Modify the **am5.xml** file by adding the Connector attribute, and supplying the encrypted passphrase to the DB resource configuration as shown in the example below:

```
XML

<Connector>
    port="8443" minSpareThreads="10"
    protocol="com.isprint.amutil.tomcat.AMCustomNIO2Protocol"
    enableLookups="false" disableUploadTimeout="true"
    keepAliveTimeout="900000" maxKeepAliveRequests="-1"
    acceptCount="100" maxThreads="200"
    scheme="https"  secure="true" SSLEnabled="true" >
    <SSLHostConfig certificateVerification="required"
```

```
sslProtocol="TLS" protocols="TLSv1.2"

ciphers="TLS_DHE_RSA_WITH_AES_256_GCM_SHA384,TLS_ECDHE_RSA_WITH_AES
_256_GCM_SHA384,

TLS_DHE_RSA_WITH_AES_256_CBC_SHA256,TLS_ECDHE_RSA_WITH_AES_256_CBC_
SHA384"
        truststoreFile="/conf/testtrust.keystore"
        truststorePassword="AesEncryptor::file=/home/am/conf-
secure/encryptor-secret.txt:LpwncJB3ODHUZVoRz/Mar97Ow3mhDkg=="
        truststoreType="JKS">
        <Certificate
certificateKeystoreFile="/conf/testserver.keystore"

certificateKeystorePassword="AesEncryptor::file=/home/am/conf-
secure/encryptor-
secret.txt:CRLmqmNktipwncJB3Owrlvo+jcz/vsoFeeHB3/gvIUQ="
            certificateKeystoreType="JKS"/>
    </SSLHostConfig>
</Connector>
```

Using HashicorpVaultEncryptor to Encrypt Database Password

> **Note:** For detailed information on configuring HashiCorp Vault to manage AccessMatrix database credentials, refer to *AccessMatrix HashiCorp Vault Implementation Guide - Using HashiCorp Vault for Database Credential Management*.

If you are using HaschiCorp Vault to manage the AccessMatrix database credentials, perform the following steps to encrypt the database password for Tomcat/DBCP:

1. Run **encryptor.bat** and type "3" to select `HashicorpVaultEncryptor`, then press the Enter key.

   The HashicorpVaultEncryptor menu is displayed.

```
- Menu: 3 - Encryptor Menu -

- EncryptorId: HashicorpVaultEncryptor


0. Back to main
1. Encrypt a passphrase
2. Enter a user salt for this encryptor
3. Generate a new random user salt
4. Use a file as user salt for this encryptor
5. Use an environment variable as user salt for this encryptor
6. decrypt
```

2. Provide a salt for the encryptor using one of the following options:

- `2. Enter a user salt for this encryptor` - Type in a user salt and press the Enter key.
- `3. Generate a new random user salt` - A random salt is generated upon selecting this option.
- `4. Use a file as user salt for this encryptor` - Type in the location of the text file and press the Enter key.

  > **Note:** If you are selecting this option, ensure that the file is in a secure directory. Restrict file access to only the OS user used to run the AccessMatrix Server and Encryptor Utility Tool.

- `5. Use an environment variable as user salt for this encryptor` - Type in the environment variable to be used as the salt and press the Enter key.

3. Back at the screen of `HashicorpVaultEncryptor`, type "1" to select `Encrypt a passphrase` and press the Enter key.
4. Provide the following values when prompted by the Encryptor Utility Tool:
   a. `Secret engine` - The secret engine used.
   Select the default option (database) by pressing Enter without supplying a value.
   b. `Role ID` - The RoleID of the HashiCorp Vault approle.
   Enter the RoleID (e.g. "fefb85d1-d835-93cb-bbd3-687fca5a12ef") and press the Enter key.
   c. `Secret ID` - The SecretID of the approle.
   Press Enter without supplying a value if you did not bind a SecretID to the approle.

   > **Note:** If you configured the approle to require the use of a SecretID, this field is mandatory.

   d. `Field name` - The field name corresponding to the database password value in the response of the HTTP GET request to the `URLto secret value`.
   Select the default option (password) by pressing Enter without supplying a value.
   e. `URL to secret value` - The URL of the configured static role in HashiCorp Vault.
   Enter the static role URL (e.g. "http://127.0.0.1:8200/v1/database/static-creds/amdb-reader") and press the Enter key.
5. The resulting ciphertext is displayed under `SingleFieldEncryptedValue`. Copy the encrypted passphrase or password under `SingleFieldEncryptedValue`.

```
- Menu: 3 - Encryptor Menu -
- EncryptorId: HashicorpVaultEncryptor
```

```
0. Back to main 1. Encrypt a passphrase 2. Enter a user salt for this encryptor 3.
Generate a new random user salt 4. Use a file as user salt for this encryptor 5. Use an
environment variable as user salt for this encryptor 6. decrypt
Enter selection (default: [0]) --> 1

Secret engine: (default: [database]) -->

Role ID: --> fefb85d1-d835-93cb-bbd3-687fca5a12ef

Secret ID: -->

Field name: (default: [password]) -->

URL to secret value: --> http://127.0.0.1:8200/v1/database/static-creds/amdb-reader

You have entered: engine=database role=fefb85d1-d835-93cb-bbd3-687fca5a12ef secret=
field=password url=http://127.0.0.1:8200/v1/database/static-creds/amdb-reader Confirm
y/n? (default: [n]) --> y
Encrypted passphrase:
Tzm4MHyYJnLjUwYMB9l3lP/q2a6Y7qHgvZHhHwRHJGJEYIjGjmLyxDrz2ol7jrneOIGDbLAL8NUjUWa2ZkP4wCu
MBGi3F/wB7Y5vEnDhG4SN/jxJrfNNdEZWq2HZwUQEQ2987fL7tk1W+cJYHLmIKGaLl7SslK7tAhHHWMU9hDEOol
uRsCVrWeb6xGyNIkYCl934JrjNUouOopfcnZrluJSm3dEFFFelakXK4oTd7Ro= EncryptorId:
HashicorpVaultEncryptor
SingleFieldEncryptedValue:
HashicorpVaultEncryptor:Tzm4MHyYJnLjUwYMB9l3lP/q2a6Y7qHgvZHhHwRHJGJEYIjGjmLyxDrz2ol7jrn
eOIGDbLAL8NUjUWa2ZkP4wCuMBGi3F/wB7Y5vEnDhG4SN/jxJrfNNdEZWq2HZwUQEQ2987fL7tk1W+cJYHLmIKG
aLl7SslK7tAhHHWMU9hDEOoluRsCVrWeb6xGyNIkYCl934JrjNUouOopfcnZrluJSm3dEFFFelakXK4oTd7Ro=
Press [Enter] to continue ...
```

6. Open the **am5.xml** file from the *<ACMATRIX_ROOT>\tomcat\conf\Catalina\localhost* folder.
7. Modify the **am5.xml** file by supplying the encrypted value to the password attribute as, shown in the example below:

XML

```
<Resource name="jdbc/DSamdb" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"
factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory"
initialSize="10" maxWaitMillis="10000"
maxTotal="80" maxIdle="20" minIdle="10" validationQuery="select 1"
username="vaultuser"
password="HashicorpVaultEncryptor:Tzm4MHyYJnLjUwYMB9l3lP/q2a6Y7qHgv
ZHhHwRHJGJEYIjGjmLyxDrz2ol7jrneOIGDbLAL8NUjUWa2ZkP4wCuMBGi3F/wB7Y5v
EnDhG4SN/jxJrfNNdEZWq2HZwUQEQ2987fL7tk1W+cJYHLmIKGaLl7SslK7tAhHHWMU
9hDEOoluRsCVrWeb6xGyNIkYCl934JrjNUouOopfcnZrluJSm3dEFFFelakXK4oTd7R
o=" driverClassName="com.mysql.jdbc.Driver"
url="jdbc:mysql://10.1.11.85:3306/amdb?useUnicode=true&amp;characte
rEncoding=utf8"
 />
```

6. Known Issues

AccessMatrix Server Slow Start-up in Linux Environment

AccessMatrix server experiences slowness during the server starts for deployment in Linux environment due to the delay in the RADIUS server start. RADIUS server pseudo-random-number generators are seeded by a call to java.security.SecureRandom. This class defaults to using "/dev/random" for the seed. It appears that "/dev/random" is either not running or has not accumulated enough random events to provide the few bytes needed by the RADIUS server at startup. This is a known issue for various Linux implementations due to the low level of available entropy for random data generation into "/dev/random". Insufficient random data in "/dev/random" causes java.security.SecureRandom used by the RADIUS server to wait until there are available/sufficient random data to be used. This issue is not consistent and the overall delay experienced by AM server start-up can range from seconds to minutes.

Mitigation

There are several ways to overcome this issue but the most recommended solution is to use rng-tools.

Kernel rng-tools

The rng-tools is a set of utilities related to random number generation in the Kernel. The main program is **rngd**, a daemon developed to check and feed random data from the hardware device to the kernel entropy pool.

This is mainly useful to increase the quantity of entropy in the Kernel to make "/dev/random" faster. By default, "/dev/random" is very slow since it only collects entropy from the device drivers and other (slow) sources. rngd allows the use of faster entropy sources, mainly hardware random number generators (TRNG) present in the modern hardware like the recent AMD/Intel processors, Via Nano or even Raspberry Pi.

While Linux itself uses the result from **TRNG** in "/dev/random", if available, these are only used as an XOR after the entropy is collected by the Kernel. So "/dev/random", by default, is slow even if you do have a **TRNG**. rngd feeds "/dev/random" itself, increasing the available entropy by far.

Installing rng-tools

1. Su to the "root" account by entering the following command:
   ```
   $ su
   ```

2. rng-tools can be installed using the following command:

   ```
   # yum install rng-tools -y
   ```

3. Configure rng-tools:

   ```
   # echo 'EXTRAOPTIONS="-i -o /dev/random -t 10 -W 2048"' > /etc/sysconfig/rngd
   ```

4. Start the rngd service:

   ```
   # service rngd start
   ```

5. Configure the rngd service to start at boot

   ```
   # chkconfig rngd on
   ```

> **Note**: "rng-tools" is installed and enabled by default in Fedora 18 and above.

Check Entropy Level

Run the following command to check the entropy level (before rngd service is started, the entropy is significantly lower):

```
# cat /proc/sys/kernel/random/entropy_avail
```

After rng-tools is installed and running, there is more entropy available to feed "/dev/random". Thus, RADIUS server start-up speed will improve and AM server start will not experience the long delay.

Issues Parsing Date and Time Formats

When upgrading AM and reusing existing implementations for reading and processing date and time formats, random characters may be included in place of whitespaces. For example, when generating the text for an email OTP which relies on processing the date and time formats within the JSON response from token/generateOTP, the text may include unwanted characters.

Due to changes introduced in JRE/JDK versions 20 and above, date and time formats generated by Java now uses a special character to represent whitespaces. This potentially affects existing parsing methods that attempt to process whitespaces within this format, since a different character is introduced in place of normal spaces.

Mitigation

For a workaround to fix issues parsing date and time formats, follow the instructions below.

1. Navigate to *<ACMATRIX_ROOT>/bin* and open the **setenv.bat** file using a code editor.
2. Search for the :setJavaOpts comment.

3. Add the following code under this comment.

```
Code

set JAVA_OPTS=%JAVA_OPTS% -
Djava.locale.providers=COMPAT
```

4. Navigate to *<ACMATRIX_ROOT>/bin* and open the **setenv.sh** file using a code editor.
5. Add the following code:

```
Code

set JAVA_OPTS=%JAVA_OPTS% -
Djava.locale.providers=COMPAT
```

6. Save the changes for both files.

Failure to Start AccessMatrix with Database Synonyms

In the AM6 series, the use of Hibernate ORM 6 introduces new restrictions on metadata validation. When Oracle databases are used, these restrictions can potentially prevent database tables from being accessed via synonyms. Subsequently, certain database tables may not be loaded and the AccessMatrix server fails to start as a result.

This issue can be identified by such errors in the **am.log** file:

```
Code

25-02-10 17:41:34.123 [main] ERROR -
(HibernatePersistenceProvider.java:228) Failed to
initialize Hibernate SessionFactory, message=Schema-
validation: missing table [am_ancestry]
org.hibernate. tool. schema.spi.
SchemaManagementException: Schema-validation: missing
table [am_ancestry]
```

To resolve this issue, perform the following steps:

1. Navigate to the directory *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/classes* and open the **amsystem.properties** file.
2. Add the **am.domain.store.default_schema** attribute and specify the name of the database schema. For example:

```
Code
```

```
am.domain.store.default_schema= amdb
```

3. Save the changes and restart the AccessMatrix server.

7. Appendices

This section contains the following appendices:

7.1. A. Summary of Hibernate SQL Dialect Classes

For certain versions of MySQL database, the Hibernate SQL dialect classes must be configured according to this table (e.g. am.domain.store.dialect=com.isprint.am.hibernate.AmMySQL57InnoDBDialect). Most of the time, you don't need to set this configuration in **amsystem.properties**.

The following table contains a summary list of Hibernate SQL dialect classes for the supported database servers.

Package Name: org.hibernate.dialect	
org.hibernate.dialect.Oracle10gDialect	An SQL dialect for Oracle 10g
org.hibernate.dialect.SQLServerDialect	An SQL dialect for Microsoft SQL Server
org.hibernate.dialect.MySQLInnoDBDialect	An SQL dialect for MySQL with InnoDB
org.hibernate.dialect.DmDialect	An SQL dialect for Dameng Database Server
com.isprint.am.hibernate.AmMySQL57InnoDBDialect	An SQL dialect specifically for MySQL 5.6, 5.7, 8.0.18 with InnoDB

7.2. B. Apache Tomcat Web Server Properties

The Apache Tomcat web server contains a number of key configuration and properties files. The configuration file is located at this location: *<ACMATRIX_ROOT>\tomcat\conf\server.xml*.

This configuration file contains all the configuration directives that determine the behaviour of the Tomcat web server. The configuration element descriptions are organized into the following major categories:

- **Top-Level Elements**
  <Server> is the root element of the entire configuration file, while <Service> represents a group of Connectors that is associated with an Engine.
- **Connectors**
  Represent the interface between external clients sending requests to (and receiving responses from) a particular Service.
- **Containers**
  Represent components whose function is to process incoming requests and create the corresponding responses. An Engine handles all the requests for a Service, a Host handles all requests for a particular virtual host, and a Context handles all requests for a specific web application.
- **Nested Components**
  Represent elements that can be nested inside the element for a Container. Some elements can be nested inside any Container, while others can only be nested inside a Context.

Restart the AccessMatrix Server after any changes are made and saved.

7.3. C. AccessMatrix Core System Properties

The AccessMatrix core system properties file is located at the following location: *<ACMATRIX_ROOT>\WEB-INF\classes\amsystem.properties*.

The following table describes the configurable properties parameters:

Properties Parameter	Description
General Properties	
ACMATRIX_HOME	The home directory of Apache Tomcat web server. The default value is " . ". **Note:** If the value is not specified, the system will initialize it to the parent directory of the current directory. At runtime, this will be changed to the canonical form of the absolute path by SystemProperties.
am.server.id	A unique AccessMatrix Server ID used to identify the instance of the AccessMatrix Server for licensing and auditing purposes during the AccessMatrix Server starts up. By default, the parameter is disabled. This parameter will only be enabled automatically if the administrator installs the AccessMatrix system through AccessMatrix Installer on Windows platform and provided the "AccessMatrix Server" component is selected for the installation. In this case, its value will be set according to the value specified by the administrator during installation. However, under certain circumstances, the administrator may be required to enable this parameter manually by setting a unique AccessMatrix Server ID.

Properties Parameter	Description
	So long as a value has been specified, the value will be used during the AccessMatrix server starts up such that if the value matches a server instance defined in the database, AccessMatrix Server will be able to start accordingly. Otherwise, the AccessMatrix Server will fail to start. Once specified, the AccessMatrix Server ID should not be changed if not necessary. **Note:** If not specified, the system will automatically set it to the local hostname of the AccessMatrix server.
am.server_monitoring.intervalInSecs	Sets the interval time of server monitoring checks in seconds. Default: 60 **Note:** The default value will always be used if the specified value is less than 60 seconds.
am.domain.id	Domain ID of the AccessMatrix server. By default, the parameter is disabled. **Note:** If not specified, the system will automatically initialize it to the Domain ID in domain store.
am.auth.session.treat-non-interactive-as-dynamic-agent	If set to "true", non-interactive user sessions will be treated as dynamic agent sessions. These non-interactive sessions will no longer expire. Default: false
JNDI-Based Datasource Connection	
am.domain.store.datasource	JNDI data source name for connecting to underlying AccessMatrix database. Typically, the value is in

Properties Parameter	Description
	following format: "jdbc/<AccessMatrix_DB_Name>" where "<AccessMatrix_DB_Name>" is to be replaced with the actual AccessMatrix database name, e.g.: "jdbc/DSamdb". For Tomcat, the value is in the form of a full datasource name, e.g.: "java:/comp/env/jdbc/DSamdb".
Ehcache	
am.ehcache.config_file	Refers to the Ehcache configuration file. Default: am5-ehcache.xml
am.ehcache.peer_discovery	There are three methods to find peers: • "db (default)" - Configure ehcache manual peer discovery using servers information in the database. • "multicast" - Configure ehcache automatic peer discovery using multicast. • "manual" - AM will not assist in setting up the Ehcache peer discovery, thus it uses the peer discovery setting in Ehcache configuration file as-is.
am.ehcache.multicast_addr	Multicast group address. (Required if peer_discovery is set to multicast.)
am.ehcache.multicast_port	Multicast group port. (Required if peer_discovery is set to multicast.)
am.ehcache.multicast_ttl	Determines how far the packets will propagate.

Properties Parameter	Description
	Options: "0" = the same host "1" (default) = the same subnet "32" = the same site "64" = the same region "128" = the same continent "255" = unrestricted
am.ehcache.ip_addr	Ehcache RMI listener IP address. Applicable when the host has multiple IPs and primary IP is not set to the correct IP. AccessMatrix server normally will automatically configure this (e.g. if the host has multiple IPs, but the primary IP is not the same as IP address configured in AM Server object). If there is a need to override the listener IP address, then this property can be set to the correct IP. (This setting is applicable only if peer_discovery=multicast/db)
am.ehcache.set_rmi_ip_addr	AccessMatrix server normally will automatically configure the Ehcache RMI listener IP address. If this property is enabled ("true"), then also ask AccessMatrix server to configure the IP address used for RMI object itself. This will actually set Java system property java.rmi.server.hostname. Default is false; however, if using AM5 bundled Tomcat, check the **setenv.bat/setenv.sh/am5w.exe** as well. This property is set to "true" in that file. (This setting is applicable only if peer_discovery=multicast/db)
am.ehcache.peer_status	If set to "true", this will enable the background thread to print out the list of peers every certain interval. Default: false

Properties Parameter	Description
am.ehcache.peer_status.interval_mins	Sets the interval for printing the discovered Ehcache peers. Default: 5 mins
am.ehcache.peer_status.list_caches	If set to "true", this will list the cache names when printing the discovered Ehcache peers. Default: false
Redis Cache	
am.ehcache.replicator.redis.channel	Specifies the channel for Redis Pub/Sub replication.
am.redis.keyPrefix	If specified, this allows for multiple, separate clusters of AccessMatrix servers to share the same database. The value specified in this property will be prefixed to the Redis Pub/Sub channel specified in am.ehcache.replicator.redis.channel.
Quartz Scheduler	
AccessMatrix uses Quartz Scheduler to handle background tasks. **Note:** Refer to Configure Quartz Scheduler for additional details.	
Token Identity Cache	
com.isprint.am.core.tokenmgmt.TokenIdentityCache.preload.tokenBatchIds	Token batch Ids for preload stress testing.
Global SSO Configuration	
am.globalsso.target.country.proxy.{x}	This attribute is used for the ApiProxyFactory. {x} is the country value. Example: am.globalsso.target.country.proxy.sg
am.globalsso.dpmoduleid	The DP Module Id of the target country. It must be the same across all participating countries.

Properties Parameter	Description
am.globalsso.proxy.global	Sets the global proxy value if it's the local server.
PKCS11	
am.crypto.pkcs11.userpin	Refers to the PKCS11 user pin.
Jcprov	
am.crypto.jcprov.<KeyProviderId>.userpin	Refers to the user pin. The key provider Id is the key provider Id set in Admin Console.
Simulated Certificate	
com.isprint.am.server.xmlrpc.XmlRpcServlet.$simCert.file	The simulated certificate file to be used for XML-RPC authentication testing. This attribute should not be enabled in UAT or production environments, instead a mutual SSL authentication with a valid agent certificate should be set up.
com.isprint.am.server.soap.WrappedWSServlet.$simCert.file	The simulated certificate file to be used for SOAP authentication testing. This attribute should not be enabled in UAT or production environments, instead a mutual SSL authentication with a valid agent certificate should be set up.
Event Manager Thread Pool	
AccessMatrix 5.0 has a thread/queue pool for event manager to dispatch event listeners like audit writer, event notifier. If high number of audit/event notification is expected or the queue busy count shown in the monitoring report is on the high side, consider tuning these properties to improve server performance. Reporting an event in AccessMatrix 5.0 is non-blocking normally. However, when the event manager queue is full, the reporting event will become blocking until: 1. The reporting thread manages to find a place in the event manager queue; 2. The blocking time exceeds the queue blocking wait time; in this case, an exception will be thrown failing the reporting thread's operation.	

Properties Parameter	Description
The administrator should tune the blocking wait time to balance the reject rate versus degraded service i.e. prefer a slower response time that may risk hanging the entire system or a quick rejection if reporting of the event is blocking.	
am.event_manager.worker_pool.max	Maximum number of worker threads allowed in the event manager thread pool. The default value is 50.
am.event_manager.worker_pool.queue.size	Queue the size that holds extra tasks whenever there are more active tasks than threads available in the event manager thread pool. The default value is 150.
am.event_manager.worker_pool.queue.blocking.wait_time_ms	Queue blocking waiting time (in millisecond) for holding each extra task in the queue in the event manager thread pool. The default value is 30000.
Message Delivery Manager	
am.delivery_manager.worker_pool.max	The number of worker threads per delivery channel. Default: 15
am.delivery_manager.worker_pool.queue.size	The maximum queue size per delivery channel. Default: 80
am.delivery_manager.worker_pool.queue.blocking.wait_time_ms	The waiting time before the reporting thread will throw an exception if the queue size becomes full.
am.delivery_manager.stopwatch.threshold_ms	The log lapse time for each event listener when the total elapsed time is above threshold (in milliseconds). Default: 2000 ms
am.delivery_manager.statistics.rejected.time_window_mins	The time window of rejected count statistics. Default: 60 mins

Properties Parameter	Description
am.delivery_manager.jmx.enabled	Setting to true will enable JMX monitoring support. Default: false
Logging Configuration	
am.log.properties	The log4j2 configuration file (XML configuration file is also supported). Default: amlog4j2.properties
Customization Configuration	
am.attr_template.file.{x}	The file containing the attribute template. {x} is a number from 0 to 99.
am.field_definition.file.{x}	The file containing the UI definition. {x} is a number from 0 to 99.
AccessMatrix Version 4 Information Related Properties	
ACMATRIX.printVersionInfo	Whether to suppress printing of AccessMatrix version 4 information as output during the server starts up. Default: false
USO Client Version Information Related Properties	
usoclient.initialversion	The initial version number of the USO Client currently bundled with the Installer. The value will only be saved to database when a new database is created. The value is based on USO Client version number, i.e. in the form of "X,X,X,X" e.g. "5,0,9,0904".
UCM Related Properties	
ucm.converter.folder	Refers to the path of the UCM Converter libraries. Default: "bin/ucmconverter/".

Properties Parameter	Description
ucm.converter.ucm2mp4.instance	Refers to the UCM Converter UCM2MP4 instance count.
Biometric Configuration	
sensetime.sdk.verifier.object.pool.instances	Set the number of instances of SenseTime FaceVerifier Object Classes to create upon start of servers. Minimum value is "1". Setting to "0" will disable all enrollment & verification feature for SenseTime. This is to allow concurrent processing. However, larger value uses more native (non Java-heap) memory.
sensetime.sdk.searcher.object.pool.instances	Set the number of instances of SenseTime FaceSearch Object Classes to create upon start of servers. Default value is "0". If value is "0", feature is disabled and 1:N verification API templateIdentifier() will not work. To enable, set to at least "1". This is to allow concurrent processing. However, larger value uses more native (non Java-heap) memory.
sensetime.sdk.antihack.object.pool.instances	Set the number of instances of SenseTime Antihack Object Classes to create upon start of servers. Default value is "0". If value is "0", feature is disabled and anti-hack feature will not work. To enable, set to at least "1". This is to allow concurrent processing. However, larger value uses more native (non Java-heap) memory.
Contextual Authentication Configuration	

Properties Parameter	Description
contextual.authn.fraud.protection.alert.types	Refers to the alert type definition for Fraud Protection.
ESSO Normalized Entitlement Configuration	
esso.task.ESSONormalizedEntitlementBackgroundTask.enabled	Setting to "true" will enable ESSO normalized entitlement background task. Default: false
AM FIDO2 Challenge	
am.fido2.challenge.persistence	If set to "true", the FIDO2 challenge will be stored in the database.
am.fido2.challenge.housekeeping.interval	Sets the interval for housekeeping the FIDO2 challenge, in minutes. Default: 60 Note: The FIDO2 challenge will only be cleaned if FIDO2 challenge persistence has been enabled.
am.fido2.challenge.expiry	Sets the expiry of the FIDO2 challenge, in minutes. Default: 5
OIDC RP Lookup Module HC4Util Configuration	
The following properties will only configure the **OIDC Relying Party Lookup Module**.	
am.hc4.oidcrplookup.pool.maxconn_per_route	The total number of connections limited to a single route (host and port). Default: 50
am.hc4.oidcrplookup.pool.maxconn_total	The total maximum number of connections available in the connection pool. Default: 100
am.hc4.oidcrplookup.pool.ttl_ms	Defines the maximum life span of persistent connections regardless of the allowed time-to-live returned by the server. Default: 180000 milliseconds

Properties Parameter	Description
am.hc4.oidcrplookup.retry.times	The maximum number of times a method will be retried. Default: 1
am.hc4.oidcrplookup.retry.request_sent_retry	Set to "true" to retry methods that have successfully sent their requests. Default: false
am.hc4.oidcrplookup.timeout.conn	Determines the timeout in milliseconds until a connection is established. Default: 10000 milliseconds
am.hc4.oidcrplookup.timeout.socket	Determines the timeout when waiting for receiving or sending data (socket read/write timeout). Default: 30000 milliseconds
am.hc4.oidcrplookup.timeout.conn_request	Maximum wait time to get a connection. This refers to the wait time for the connection pool to return a connection. Default: 30000 milliseconds
am.hc4.oidcrplookup.socket.receive-buffer-size	Sets the size of the receive buffer for newly created sockets.
am.hc4.oidcrplookup.socket.send-buffer-size	Sets the size of the send buffer for newly created sockets.
am.hc4.oidcrplookup.socket.keep-alive	Indicate whether or not to have socket keep alive turned on.
am.hc4.oidcrplookup.socket.linger	Sets the information about the linger state of the associated socket.
am.hc4.oidcrplookup.socket.timeout	Defines the socket timeout in seconds, which is the time waiting for data - after establishing the connection. It is the maximum period of inactivity between two consecutive data packets. This setting is the same as

Properties Parameter	Description
	am.hc4.oidcrplookup.timeout.socket.
am.hc4.oidcrplookup.socket.tcp-no-delay	Sets the socket TCP_NODELAY flag.

7.4. D. AccessMatrix Admin Console Properties

The AccessMatrix Admin Console properties file is located at the following location:

<ACMATRIX_ROOT>\WEB-INF\classes\wpe_config.properties.

The following table describes the configurable properties parameters:

Properties Parameter	Description
Advanced Features Enablement Properties	
advancedFeatures.enable	Whether to enable the advanced features. Advanced features include the selection of an authentication scheme used to log in to AccessMatrix Server, as well as segment specification pertaining to the login administrator. Valid value is either "true" or "false". Set the value to "false" means disable the advanced features. The default value is false.
Authentication Scheme Properties	
authentication.schemes	The authentication schemes available for selection. The authentication schemes must be currently supported by the Admin Console for login to AccessMatrix Server. Each authentication scheme must be separated by " , ". The administrator may specify " * " to list all authentication schemes currently defined in the system. Several pre-defined authentication schemes available for selection are: • "SystemRealm" • "40172" • "40170" • "ADVascoComposite"
default.authentication.scheme	The default authentication scheme used to log in to AccessMatrix Server. Default: SystemRealm.
idm.entitlementPolicy.segment	Setting this to "true" will display the default segment in UIM.
certificatelogin.url	Refers to the certificate challenge URL that would challenge user with a client certificate authentication. e.g. "certificatelogin.url=https://testserver:8443/am5"

Properties Parameter	Description
certificatelogin.cookie.domain	(Optional) Refers to the cookie domain setting.
certificate.challenge.scheme	Refers to the authentication realm(s) that will be used for certificate login. Multiple values are allowed, separated by comma. e.g. "certificate.challenge.scheme=40153"
Autocomplete Login Fields	
autocomplete.login.fields	If set to “true”, this enables autocompletion of login fields in Admin Console using keyboard caching. By default, this is set to "false" to disable the autocompletion of login fields, regardless of the web browser’s keyboard caching settings.
Language Properties	
language.selections	The languages available for selection. The languages must be currently supported by the Admin Console. Each language must be separated by ",". Several currently supported languages available for selection are: • en_US(English) • zh_CN(\u7B80\u4F53\u4E2D\u6587) • zh_TW(\u7E41\u9ad4\u4E2D\u6587) • ja_JP(\u65E5\u672C\u8A9E) • th_TH(Thai)
language.default	The default language used by the Admin Console. The default value is "en_US".
authentication.showGeneralFailMsg.codes	Specifies the list of authentication exception codes for general fail message: "Wrong id and password" will be displayed as the error message. e.g. "authentication.showGeneralFailMsg.codes=10001,10020,10004"
Search User	
search.user.mobile-required	Setting this to "true" will allow searching of user by user's mobile attribute. e.g. "search.user.mobile-required=false"
search.user.email-required	Setting this to "true" will allow searching of user by user's email attribute.

Properties Parameter	Description
Segment	
tiu.create.segment.required	Setting this to "true" will display the Segment field in the Create Token Group of the Token Import Utility (TIU).
Workflow Request	
load.workflowRequest.for.all.OpDashBoard	Setting this to "true" will load the workflow request count when clicking all the menus under **Operation Dashboard**. If this is set to "false", then the workflow request count will only be loaded for the **Request** menu (**Operation Dashboard > Request**). The default value is true.
workflowRequest.count.limit.OpDashBoard	Sets the query limit for the count of **Pending my approval** workflow requests in the **Operation Dashboard**. The default value is 50 and the maximum value is 200.
workflowRequest.eachQuery.maximum.record	Sets the maximum number of records for each query of **Pending my approval** workflow requests. The default value is 50.
workflowRequest.page.minimum.record	Sets the minimum number of records to be returned for Pending my approval workflow requests. If the records returned are less than the specified minimum value, then the AccessMatrix server will automatically query the next set of records until it reaches the specified minimum or until the system queries up to 5 times. The default value is 5.

7.5. E. AccessMatrix Ehcache Configuration

AccessMatrix has **am5-ehcache.xml**, which has preconfigured all the AccessMatrix caches. You may refer to this section if you need to make changes on the cache configuration.

The AccessMatrix Ehcache configuration file is located in: *<ACMATRIX_ROOT>\WEB-INF\classes\am5-ehcache.xml*.

E.1 Cache Configuration Explanation

To configure cache, the following attributes are required:

Properties Name	Description
name	Sets the name of the cache. This is used to identify the cache. It must be unique.
maxElementsInMemory	Sets the maximum number of objects that will be created in memory.
eternal	Determines whether elements are eternal. If eternal, timeouts are ignored and the element never expires.
overflowToDisk	Determines whether elements can overflow to disk when the memory store has reached the maxInMemory limit. For AccessMatrix, this should be set to false.

The following attributes are optional:

Properties Name	Description
timeToIdleSeconds	Sets the time to idle for an element before it expires, i.e. the maximum amount of time between accesses before an element expires is only used if the element is not eternal. A value of "0" means that an element can idle for infinity. Default value is 0.
timeToLiveSeconds	Sets the time to live for an element before it expires, i.e. the maximum time between the creation time and when an element expires. It is only used if the element is not eternal. A value of "0" means that an element can live for infinity. Default value is 0.

Properties Name	Description
maxElementsOnDisk	Sets the maximum number of objects that will be maintained in the DiskStore. Default value is 0, meaning unlimited.
diskPersistent	Indicate whether the disk store persists between restarts of the virtual machine. Default value is false.
diskExpiryThreadIntervalSeconds	The number of seconds between runs of the disk expiry thread. Default value is 120 seconds.
memoryStoreEvictionPolicy	The policy would be enforced upon reaching the maxElementsInMemory limit. The default policy is Least Recently Used (specified as LRU). Other policies available - First In First Out (specified as FIFO) and Less Frequently Used (specified as LFU).

Cache elements can also contain sub-elements which take the same format of a factory class and properties. Defined sub-elements are:

- cacheEventListenerFactory - Enables registration of listeners for cache events, such as put, remove, update, and expire.
- bootstrapCacheLoaderFactory - Specifies a BootstrapCacheLoader, which is called by a cache on initialisation to prepopulate itself.

Configure cacheEventListenerFactory

Each cache that will be distributed, needs to set a cache event listener which replicates messages to the other CacheManager peers. For the built-in RMI implementation this is done by adding a cacheEventListenerFactory element of type RMICacheReplicatorFactory to each distributed cache's configuration, as per the following example:

XML

```
<cacheEventListenerFactory
class="net.sf.ehcache.distribution.RMICacheReplicatorFactory"
    properties="replicateAsynchronously=true,
                replicatePuts=true,
                replicateUpdates=true,
                replicateUpdatesViaCopy=true,
                replicateRemovals=true,
```

```
asynchronousReplicationIntervalMillis=100"/>
```

The RMICacheReplicatorFactory recognizes the following properties:

Properties Name	Description
replicatePuts	Indicates whether the new elements placed in a cache are replicated to others. Default value is true.
replicateUpdates	Indicates whether the new elements which override an element already existing with the same key are replicated. Default value is true.
replicateRemovals	Indicates whether element removals are replicated. Default value is true.
replicateAsynchronously	Indicates whether replications are asynchronous or synchronous. Default value is true.
replicateUpdatesViaCopy	Indicates whether the new elements are copied to the other caches, or whether a remove message is sent. Default value is true.
asynchronousReplicationIntervalMillis	The asynchronous replicator runs at a set interval of milliseconds. The default value, when not set, is 1000 (ms). The minimum value is 10 (ms). This property is only applicable if replicateAsynchronously is set to "true". For AccessMatrix, since version 5.6.4, AccessMatrix sets this value to 100 ms for all AccessMatrix-related caches. If you need to make further modification/tweaking, you may do so by modifying the value in the respective caches.

Configure bootstrapCacheLoaderFactory

The RMIBootstrapCacheLoader bootstraps caches in clusters where RMICacheReplicators are used. It is configured as per the following example:

XML

```
<bootstrapCacheLoaderFactory

class="net.sf.ehcache.distribution.RMIBootstrapCacheLoaderFactory"
```

```
    properties="bootstrapAsynchronously=true,
maximumChunkSizeBytes=5000000"/>
```

The RMIBootstrapCacheLoaderFactory recognises the following optional properties:

- bootstrapAsynchronously - Indicates whether the bootstrap happens in the background after the cache has started. If set to false, bootstrapping must complete before the cache is made available. Default value is true.
- maximumChunkSizeBytes - Caches can potentially be very large, larger than the memory limits of the VM. This property allows the bootstrapper to fetched elements in chunks. Default chunk size is 5000000 (5MB).

Configure JMSCacheManagerPeerProviderFactory

The list below shows the actual properties extracted from the decompiled JMSCacheManagerPeerProviderFactory class.

Properties Name	Description	Usable in Apache ActiveMQ? (Yes\No)
initialContextFactoryName (mandatory)	The name of the factory used to create the message queue initial context.	Yes
providerURL (mandatory)	The JNDI configuration information for the service provider to use.	Yes
replication TopicConnectionFactoryBindingName (mandatory)	The JNDI binding name for the TopicConnectionFactory.	Yes
replication TopicBindingName (mandatory	The JNDI binding name for the topic name.	Yes
securityPrincipalName	The JNDI java.naming.security.principal.	No
securityCredentials	The JNDI java.naming.security.credentials.	No
urlPkgPrefixes	The JNDI java.naming.factory.url.pkgs.	No
userName	The user name to use when creating the TopicConnection to the Message Queue.	Yes

Properties Name	Description	Usable in Apache ActiveMQ? (Yes\No)
password	The password to use when creating the TopicConnection to the Message Queue.	Yes
acknowledgementMode	The JMS Acknowledgement mode for both publisher and subscriber. The available choices are" • "AUTO_ACKNOWLEDGE" • "DUPS_OK_ACKNOWLEDGE" • "SESSION_TRANSACTED". The default is AUTO_ACKNOWLEDGE.	Yes
listenToTopic	True or false. If "false", this cache will send to the JMS topic but will not listen for updates. Default is true.	Yes
getQueueConnectionFactoryBindingName (mandatory)	The JNDI binding name for the TopicConnectionFactory.	Yes
getQueueBindingName (mandatory)	The JNDI binding name for the topic name.	Yes

E.2 List of Caches

Category	Cache Names	Description
Hibernate caches	com.isprint.am.core.auth.UserAuthState	The cache helps to speed up repetitive queries, but frequent modification of the same object may cause stale modification exception when replication is not fast enough.
	com.isprint.am.core.auth.UserSession	
	com.isprint.am.infra.BaseObject	
	com.isprint.am.infra.Attribute	
	com.isprint.am.core.impl.TwoWayRelation	
AM Main Objects	com.isprint.am.server.BaseServerContext.mainObjects	The cache stores Global Setting, Server objects and
	com.isprint.am.server.BaseServerContext.moduleHandlers	

Category	Cache Names	Description
	com.isprint.am.core.impl.ServerRepository.serverIdIntegerUUID	Module handler instances. It should not be modified.
Cached Object Repository	com.isprint.am.core.impl.CachedObjectRepository.*	This is used to store policies and configurations (such as SAML Identity Provider, UAM Application), groups, and relations. In most cases, there is no need to modify these caches.
Short-lived User cache	com.isprint.am.core.impl.UserRepository.principals	This is used as part of certain processing to validate or compile other data (such as ESSO entitlements, OAuth user identity). The cache has short lifespan.
RADIUS	com.isprint.am.core.auth.EhCacheSessionRepository.sessionCache	This is used by AccessMatrix as RADIUS Server to cache RADIUS session when challenge response protocol is used
USO	com.isprint.am.core.esso.ESSORepository.entitlements	This cache contains User/Group - ESSO Application entitlements.
UAS	com.isprint.am.core.auth.E2EESessionRepository.sessionCache	E2EE Authentication and Data Protection session caches. This cache is used to stored the E2EE session caches. The cache contents are replicated to allow

Category	Cache Names	Description
		E2EE session to be completed in other servers. AccessMatrix sets cache replication interval to 100 ms which normally works well for most cases of user E2EEA password entries and E2EE Data Protection data entries.
	MemoryTokenStore	Stores SMS-OTP tokens for default MemoryTokenStore. It is important to set the cache's timeToLiveSeconds property to be slightly longer than OTP Expiry Policy. Default value is 900 seconds.
	com.isprint.am.core.tokenmgmt.fido.FIDORequestChallengeRepository.FIDORequestChallengeCache	FIDO authentication request session. The cache contents are replicated to allow Authorization Code exchange to be completed in other servers.
Others	com.isprint.am.core.event.EventRepository.deliveryResult	Cache is used to store message delivery result when querying its status is required.
	com.isprint.am.core.impl.EhCacheJobRepository.jobCache	Cache is used to store bulk job operation such as bulk token deletion or token group reassignment.

Category	Cache Names	Description
	com.isprint.am.report.JasperReportPrintRepository. jasperReportPrintCache	Cache is used to store Admin Console report result.
UAM - SAML	com.isprint.am.core.saml.SAMLRepository.spIssuer	This cache stores SAML SP Issuer to SAML SP UUID identification.
UAM - OAuth/OIDC	com.isprint.am.core.oauth.OAuthRepository.clientId	This cache stores OAuth client_id to OAuth Client identification.
	com.isprint.am.core.oauth.OAuthStorage.accessToken	This cache stores OAuth access_token. Access Token is stored in database. This cache is to speed up retrieval and validation check.
	com.isprint.am.core.oauth.OAuthStorage.refreshToken	This cache stores OAuth refresh_token. Refresh Token is stored in database. This cache is to speed up retrieval and validation check.
	com.isprint.am.core.oauth.OAuthStorage.authorizationCode	This cache stores OAuth Authorization Code. The cache contents are replicated to allow Authorization Code exchange to be completed in other servers.
UAM - JWT	com.isprint.am.core.jwt.JWTAuthnRepository.clientId	JWT client Id cache for identification

Category	Cache Names	Description
		when Authentication API JWT generation is used.
CLP	com.isprint.am.core.impl.CommonLoginPageCacheRepository.CLPNonceCache	USO authentication session via CLP.
Session	com.isprint.am.core.auth.InvalidatedSessionCacheRepository	This cache stores invalidated session to allow AccessMatrix to check for session validity when session persistence is not enabled.

7.6. F. Session Counter Plugin

This is an additional feature for controlling the maximum concurrent sessions per authentication realm. The way it works is by counting the existing sessions when the AccessMatrix server runs for the first time. Every time a user login or log out, this session counter will also be updated. There is a recounting scheduled job that will run during off-peak hours to refresh the session counter.

To enable this feature for a specific realm, a custom attribute called **maxConcurrentSessionCount** must be created in the respected realm. If total login sessions exceed a specified value for a specific realm, then the new login requests for that realm will be rejected.

It is a plugin, implementing a custom login policy. This feature will only work with Session Persistence enabled (i.e. login sessions are persisted to database), regardless of the selected session concurrency option. Navigate to **Administration Dashboard > Configuration > Global Setting** and go to the **Session tab > Concurrent Session Policy** section to check the existing configuration.

> **Note**: The following sections present the steps for deploying the session counter plugin using AccessMatrix's bundled Apache Tomcat server.

Deployment of Plugin

1. Go to *<ACMATRIX_ROOT>\sdk\sessioncounter* directory.
2. Copy **am-sessioncounter-plugin.jar** to *<ACMATRIX_ROOT>\tomcat\plugins* folder.
3. Copy **am-sessioncounter-job.jar** to *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\lib* folder.

Enable Session Counter Cache

1. Go to *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes* directory and open **am5-ehcache.xml**.
2. Uncomment com.isprint.am.core.auth.ext.SessionCounter to enable cache.

XML

```
    <!-- SessionCounter cache -->

    <cache name="com.isprint.am.core.auth.ext.SessionCounter"
        maxElementsInMemory="10000"
        eternal="false"
        overflowToDisk="false"
        timeToLiveSeconds="108000"
```

```
        timeToIdleSeconds ="0">
        <cacheEventListenerFactory
class="com.isprint.am.core.cache.ehcache.CacheReplicatorFactory"
            properties="replicateAsynchronously=true,
                        replicatePuts=true,
                        replicateUpdates=true,
                        replicateUpdatesViaCopy=true,
                        replicateRemovals=true,
                        asynchronousReplicationIntervalMillis=100
"/>
        <bootstrapCacheLoaderFactory

class="com.isprint.am.core.util.AmBootstrapCacheLoaderFactory"

properties="am.factoryClass=net.sf.ehcache.distribution.RMIBootstra
pCacheLoaderFactory, am.stage=AfterAutomaticStartOfModuleHandler,
bootstrapAsynchronously=false, maximumChunkSizeBytes=5000000"/>
    </cache>
```

Note: Uncommenting this session counter cache will also enable bootstrapCacheLoaderFactory, an ehcache setting to enable cache replication during bootstrap. This is used for multi-instance deployment of AccessMatrix server.

Configure Schedule Job

A scheduled job will run during off-peak hours to refresh the counting.

Note: This configuration is optional. You may add this setting if you want to set the execution time of the scheduled job (SessionCounterJob). If this is not set, this job will run every 00:00 (midnight).

1. Go to *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes* directory and open **amsystem.properties**.
2. Add a custom cron expression for SessionCounterJob using sessioncounter.job.cronexp as the key.

   Example: sessioncounter.job.cronexp=0 10 16 ? * * *

   Note: This cron expression means the session counter job will run every 4:10PM everyday.

Add Session Counter Custom Login Policy Handler

1. Go to *<ACMATRIX_ROOT>\sdk\sessioncounter\res* directory.
2. Copy the **wpecustom-sessioncounter.properties** and **field-definitions-sessioncounter.xml** files to *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes* directory.

3. Rename **wpecustom-sessioncounter.properties** to **wpecustom.properties**. If there's already an existing **wpecustom.properties** in this directory, then add the content of **wpecustom-sessioncounter.properties** to the existing **wpecustom.properties** file.
4. On the same location, open **amsystem.properties** and look for Customization Configuration.
5. Uncomment am.field_definition.file.{x} and specify the following value:

```
XML

am.field_definition.file.0=field-definitions-
sessioncounter.xml
```

6. Save the changes.
7. Start AccessMatrix server.

Configure Login Policy

1. Navigate to **Administration Dashboard > Security Policy > Authentication > Login Policy**.
2. Click the **Create** button and select "Max Concurrent Session Per Realm Login Policy" from the **Choose Login Policy Implementation** page.
3. Click the **Next** button and the **Create Login Policy** page will be displayed.
4. Enter the login policy details and click **Save**.

Attribute Name	Description
*Login Policy Id	Unique login policy Id.
*Login Policy Name	Name of login policy.
Description	Description of login policy.
Version	Login policy entry version.
Implementation	Type of login policy implementation, i.e. "Max Concurrent Session Per Realm Login Policy".
Attributes Tab	
Login Account	
Check login account status	If set to "true", then login account status will be checked and will display corresponding error message if counter already reached the set bad login count. Otherwise, user will be allowed to login if correct password is entered regardless if login account status was already

Attribute Name	Description
	disabled. Default: true
Disable login account if bad login count reaches	Sets the maximum bad login count. If bad login count reaches the specified number, then login account will be disabled. Default: 3
Allow resetting password without force password change	Allows password reset without forcing a password change. **Note:** This feature is applicable only if using API and not with Admin Console as the **Force Change Password** user attribute is always enabled in Admin Console. Default: false
Allow enabling of login account without resetting password	Allows login account activation without resetting the password. **Note:** This attribute is applicable only for the **Modify Account** button, which is hidden by default. This button can change the login account's status. Uncomment the Modify Account button tag in **field-definitions.xml** to display the button. Default: false
Increment bad login count for the wrong password used during change password	Specifies the action to be done if change password failed due to an incorrect password. • "Do not increment" - No action to be taken. • "Increment same bad login counter as login" - Increment the bad login count if change password failed due to an incorrect password. • "Increment separate bad login counter" - This will create or increment a separate counter if change password failed due to an incorrect password. Default: Do not increment
Auto Unlock	
Enable Auto Unlock By Time	If enabled, then locked accounts will be unlocked after the specified time in the **Auto Unlock Time Duration** attribute.
Auto Unlock Time Duration (in minutes)	Sets the time duration when auto-unlock will be executed.
Others	
Include new password in change and reset password event	Setting this to "true" will include the new password in event object for successful change and reset password events.

Configure Login Module

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module**.
2. Create a new login module. Alternatively, you may edit the login module of the authentication realm to be controlled (if using pre-created login module), e.g. SystemPasswordBasic.
3. Set the **Login Policy** value as the login policy created in the previous section.
4. Save the changes.

Configure Authentication Realm

1. Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm**.
2. Create a new authentication realm or you may click on any pre-created realms, e.g. "40151".
3. For example, if using authentication realm 40151, click on the **Edit** button. The **Edit Authentication Realm** page is displayed.
4. Ensure that the login module value is using the configured login module in previous section to implement the policy on the selected realm.
5. Click the **Add Custom Attribute** button and provide the following values:

Attribute Name	maxConcurrentSessionCount
Data Type	Integer
Attribute value	The maximum concurrent session count value for this authentication realm.

6. Save the changes.

Assign Login Accounts to the User

User is an entity that accesses the system and may belong to a default store (local repository) or an external user store, e.g. AD (Active Directory). Assign the configured authentication realm to the user by adding it to **Login Accounts** tab. Save the changes.